Multi-Class Imbalanced Data Handling with Concept Drift in Fog Computing: A Taxonomy, Review, and Future Directions

FARHANA SHARIEF, Department of Software Engineering, University of Sargodha, Sargodha, Pakistan
HUMAIRA IJAZ, Computer Science & IT, University of Sargodha, Sargodha, Pakistan
MOHAMMAD SHOJAFAR, 5G & 6G Institute for Communication Systems, University of Surrey, Guildford, United Kingdom of Great Britain and Northern Ireland
MUHAMMAD ASIF NAEEM, Department of Computer Science, National University of Computer and Emerging Sciences, Islamabad, Pakistan

A network of actual physical objects or "IoT components" linked to the internet and equipped with sensors, electronics, software, and network connectivity is known as the Internet of Things (IoT). This ability of the IoT components to gather and share data is made possible by this network connectivity. Many IoT devices are currently operating, which generate a lot of data. When these IoT devices started collecting data, the cloud was the only place to analyze, filter, pre-process, and aggregate it. However, when it comes to IoT, the cloud has restrictions regarding latency and a more centralized method of distributing programs. A new form of computing called Fog computing has been proposed to address the shortcomings of current cloud computing. In an IoT context, sensors regularly communicate signal information, and edge devices process the data obtained from these sensors using Fog computing. The sensors' internal or external problems, security breaches, or the integration of heterogeneous equipment contribute to the imbalanced data, i.e., comparatively speaking, one class has more instances than the other. As a result of this data, the pattern extraction is *imbalanced*. Recent attempts have concentrated heavily on binary-class imbalanced concerns with exactly two classes. However, the classification of multi-class imbalanced data is an issue that needs to be fixed in Fog computing, even if it is widespread in other fields, including text categorization, human activity detection, and medical diagnosis. The study intends to deal with this problem. It presents a systematic, thorough, and in-depth comparative analysis of several binary-class and multi-class imbalanced data handling strategies for batch and streaming data in IoT networks and Fog computing. There are *five* major objectives in this study. First, reviewing the Fog computing concept. Second, outlining the optimization metric used in Fog computing. Third, focusing on binary and multi-class batch data handling for IoT networks and Fog computing. Fourth, reviewing and comparing the current imbalanced data handling methodologies for multi-class data streams. Fifth, explaining how to cope with the concept drift, including novel and recurring classes, targeted optimization measures, and evaluation tools. Finally, the best performance metrics and tools for concept drift, binary-class (batch and stream) data, and multi-class (batch and stream) data are highlighted.

This work is partly supported by EU HORIZON-TMA-MSCA-SE project TRACE-V2X under grant No. 101131204.
Authors' Contact Information: Farhana Sharief, Department of Software Engineering, University of Sargodha, Sargodha, Punjab, Pakistan; e-mail: farhana.shareef@uos.edu.pk; Humaira Ijaz, Computer Science & IT, University of Sargodha, Sargodha, Pakistan; e-mail: humaira.bilalrasul@uos.edu.pk; Mohammad Shojafar, 5G & 6G Institute for Communication Systems, University of Surrey, Guildford, Surrey, United Kingdom of Great Britain and Northern Ireland; e-mail: m.shojafar@surrey.ac.uk; Muhammad Asif Naeem, Department of Computer Science, National University of Computer and Emerging Sciences, Islamabad, Pakistan; e-mail: asif.naeem@nu.edu.pk.

ACM 0360-0300/2024/10-ART16
https://doi.org/10.1145/3689627

CCS Concepts: • **Computing methodologies → Supervised learning by classification**; **Artificial intelligence**; • **General and reference → Surveys and overviews**; • **Information systems → Data streams;**

Additional Key Words and Phrases: Cloud computing, fog computing, Internet of Things (IoT), multi-class imbalanced data stream, concept drift

ACM Reference Format:
Farhana Sharief, Humaira Ijaz, Mohammad Shojafar, and Muhammad Asif Naeem. 2024. Multi-Class Imbalanced Data Handling with Concept Drift in Fog Computing: A Taxonomy, Review, and Future Directions. *ACM Comput. Surv.* 57, 1, Article 16 (October 2024), 48 pages. https://doi.org/10.1145/3689627

1 Introduction

The **Internet of Things (IoT)** is a vast and heterogeneous landscape being emerged as the next computing paradigm that will undoubtedly revolutionize how we interact and conduct business by connecting billions of devices, objects, and living things to the Internet. This network has widely dispersed, intelligent, tiny, self-configurable devices with limited processing and storage capacities, which can cause problems with performance, security, privacy, and reliability [1]. It benefits various application sectors, including smart buildings, healthcare, manufacturing, and many more. These interconnected IoT components generate a wide range and massive amounts of data. The IoT components generate over 2.5 quintillion bytes of data daily. [2]. Estimates suggest 45.41 billion connected IoT components will be connected by 2023, [3], rising to 1.2 trillion by 2030 [4].

1.1 IoT Data Types

A wide variety of applications and environment that IoT components operate in is reflected in diverse spectrum of data types that these devices create. It is crucial to understand these data types for fully utilizing IoT technology. Therefore, these IoT devices generate data about the following features [5]:

(1) **Status Data:** IoT status data is the most prevalent and fundamental type of data. It serves as a starting point for more complex investigations, such as determining whether a certain unit component is functioning. Almost anything will generate data like this. Therefore, it serves as a baseline.
(2) **Location Data:** It is the information about a device's or other asset's unique geographical whereabouts that is gathered and tracked by GPS satellites in a specific network. It is an extension of GPS because, in many congested areas, GPS does not work.
(3) **Automation Data:** It is unavoidable and is used to change the current state of the system. Manufacturers of smart lights, for example, use sensor data to direct the store managers in the opening of checkout lines.
(4) **Actionable Data:** It is similar to status information with a follow-up strategy. A dashboard alert indicating server downtime, accompanied by a recommended reboot procedure to restore service.
(5) **Feedback Loop with IoT Data:** It is establishing a feedback loop from the client to the developer to assess real-world behavior while preserving appropriate levels of privacy, security, and anonymity.

1.2 Analytics-driven Intelligence in the Internet of Things

This diverse and enormous IoT dataset is analyzed using IoT analytics, which offers insightful data. IoT analytics adds value to this data by fetching, combining, and evaluating it. This procedure encourages innovation across a range of industries, enhances functional performance, and allows

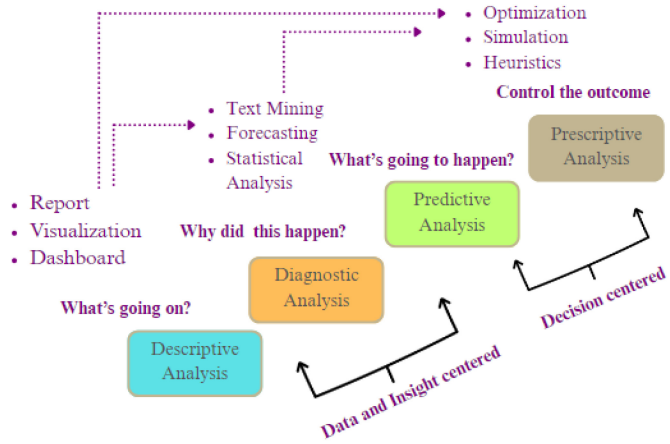

Fig. 1. Analytics taxonomy.

for better informed decision-making. Among the many tasks carried out by IoT analytics are the anticipatory maintenance of equipment, the efficient use of resources, enhanced customer experiences, and the creation of new products. Moreover, real-time responses to dynamic conditions are provided via IoT data analytics. To put it simply, IoT data analytics is essential to realizing the full potential of IoT technology and transforming data into a strategic asset that can advance both enterprises and society. Furthermore, individuals and businesses may benefit from data analytics. The taxonomy of data analytics is shown in Figure 1 with the following categories:

(1) Descriptive analysis is used to examine historical data. It, for example, employs data-mining techniques to find patterns and establish connections.
(2) Diagnostic analysis is used to identify the causes of events as well as potential issues and failures, for example.
(3) Predictive analysis employs previous data to forecast data patterns. In the production process, for example, consumer behavior forecasts are crucial.
(4) Prescriptive analysis takes all the other types' results and applies them to making the best judgments possible to obtain a predictable outcome.

1.3 Cloud-based Processing and Analytics

In the beginning, the IoT analytics performed by a centralized cloud-based architecture is known as CIoT [6]. In this paradigm, the IoT can benefit from the cloud's resources and limitless capabilities. This architecture has only two tiers. The first tier is the end-user devices that are using cloud services. The second tier is the cloud. A business model called cloud computing provides essential network connectivity in several forms, including storage, services, and networks. It also has virtually infinite processing and storage capacity. Although the CIoT has been a successful platform for many IoT applications, the unlimited increase in IoT applications generates an endless storm of data that Cloud servers cannot process alone. Furthermore, these IoT applications now also need location awareness, low latency, geo-distribution, and mobility, [7] due to technological advancements and a new wave of internet deployment adding more to data generated by these IoT applications. Transferring this sheer amount of data to distant cloud servers consumes heavy bandwidth but causes delays that are not tolerable by many real-time applications. There are restrictions on how much data can be transferred to the cloud [8].

The emerging technologies to handle these challenges in the cloud include volunteer computing, software-defined computing, mobile computing, and Fog and edge computing. According to a survey [9], Fog computing is the most common paradigm for analyzing IoT data and giving network devices cloud functionalities.

1.4 Fog Computing Paradigm

A distributed computing technique known as Fog computing emerges as a solution to the bandwidth and latency issues that cloud-centric IoT systems bring about. This technique uses the cloud as a link to connect to devices at the edge. Retaining content closer to the edge improves the capabilities of cloud-based services. By offering decentralized computing services, Fog computing enables local data processing, lowers latency and bandwidth consumption, which improves productivity and service quality. This approach works well for time-sensitive applications that require immediate analysis and actions. It is therefore a more effective and responsive computing model that reduces dependency on centralized data centers. Smart health management, smart buildings, smart grids, and smart manufacturing are a few of the most popular uses of Fog computing.

1.5 Fog Data Life Cycle

Combining IoT data analytics with Fog computing allows efficient processing and analysis of the large IoT dataset. Fog computing leverages data analysis to enable real-time insights and decision-making without the latency associated with cloud computing, which entails processing massive amounts of data created by IoT devices at or near the edge of the network. The Fog data analysis lifecycle is the sequence of events that data goes through in the Fog computing architecture, including the data's initial generation by IoT devices, processing and analysis at the edge, and final use for decision-making or additional aggregation for cloud storage or in-depth analysis.

Fog computing, improving decision-making. The Fog data analysis comprises three layers. There are three layers of Fog data analysis. In the first layer, the data is gathered from IoT components and sensors before being sent to the Fog layer. This layer contains actuators for command execution coming from the above layer. The subsequent Fog layer comes after that. It consists of two sub-layers. The first sub-layer, the Fog-device Fog sub-layer, handles the physical devices' routines, protocol interpretation, signal de-noising, authentication, and data storage. Additionally, this layer conducts local decision-making and light analysis. The Fog-cloud sub-layer is the other Fog sub-layer. This sub-layer handles compression and decompression as well as encryption and decryption. The third layer is the cloud layer, which transfers aggregated data. It stores data permanently and makes global decisions. After processing and analyzing the incoming query, it generates feedback and sends it to the Fog layer. A detailed **Fog Data Analysis (FDA)** model proposed by Reference [10] addresses various challenges such as heterogeneous Fog network, quality of service, programming model and interface, resource management, security, and privacy. Figure 2 shows the basic structure of FDA.

However, the occurrence of imbalanced data is a major challenge, especially in settings where timely and accurate insights from data are crucial for decision-making in Fog data analytics, particularly in IoT and Fog computing environments. When there is an uneven distribution of data among various classes, some types of data predominate over others, which results in this condition. Analytics models may be negatively impacted by this imbalance, which can lead to a bias in favor of the more commonly represented classes and, as a result, reduce their ability to predict important but rare events. Such biased analytics have a far greater negative impact than just inaccurate analysis: They fundamentally compromise the quality of decision-making processes. Similar to how unbalanced data prevents systems from detecting threats in security systems,

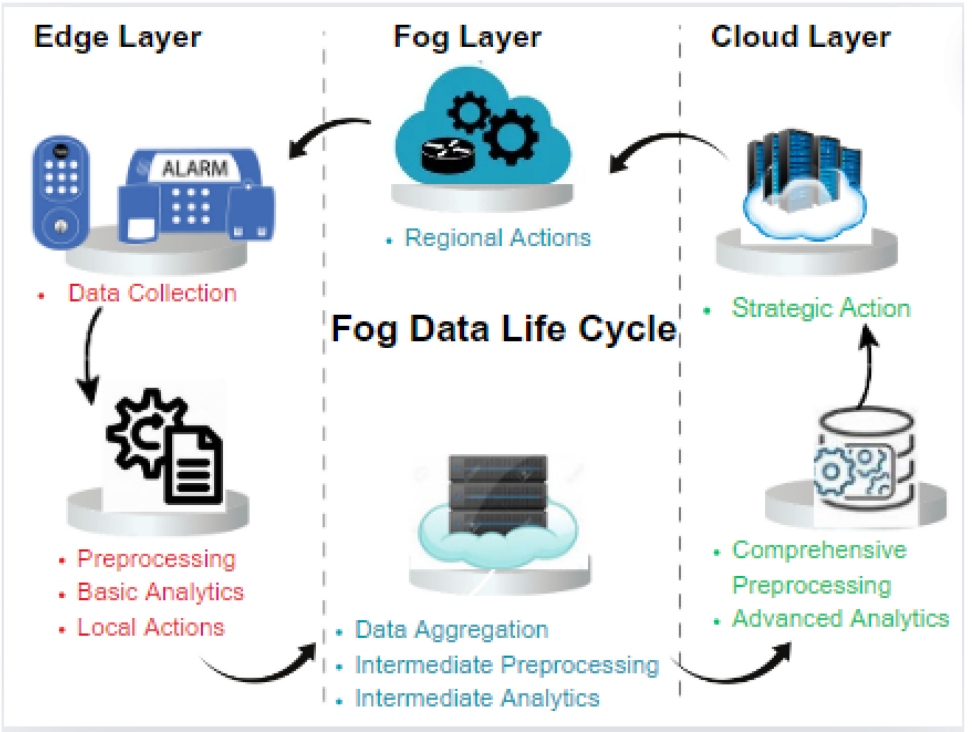

Fig. 2. Fog data life cycle.

unbalanced data prevents scenarios needing predictive maintenance for system integrity from detecting early signs of system malfunction. Furthermore, excessive caution or overstretching in areas that do not require immediate attention can result from this imbalance, which can lead to inefficient resource allocation. Moreover, the challenge of data imbalance in fog analytics extends to the optimization of the fog computing infrastructure itself. It can hinder the system's ability to effectively manage load distribution, energy consumption, and bandwidth usage, affecting the overall performance and sustainability of the fog computing environment. Therefore, it is crucial to address data imbalance to enhance the reliability of Fog analytics. Synthetic data generation, data augmentation, and a few advanced machine learning techniques handle imbalanced data. Resolving this imbalance is essential to guaranteeing that data analytics in fog environments can produce trustworthy, actionable insights that facilitate prompt and well-informed decision-making.

1.6 Reasons of Imbalanced Data in IoT, Fog Computing, and WSN

Technical, environmental, and operational issues could cause data imbalance. We elaborate on these causes below, offering a thorough rundown of the elements causing data imbalances in Fog computing:

— **Heterogeneity of Devices**
Each heterogeneous IoT component generates data at varying rates and formats, leading to imbalanced data distribution. In a smart city, for instance, traffic cameras may continuously collect data, but environmental sensors may only collect within certain conditions leading to imbalanced data.

— **Rareness of Event**
Detecting rare but critical events leads to datasets where the number of such occurrences is much greater than that of typical events. For instance, because irregular heart rates are uncommon relative to normal heart rates, health monitoring systems may have trouble detecting them.
— **Constantly Changing Network Topology**
Frequent connection and disconnection of devices in IoT environments cause network instability. The energy constraints arising from limited battery life of devices, mobility of wearable devices introducing variability in data capture and network connectivity, and other environmental factors like temperature fluctuations, cause the network topology to change constantly. This dynamic nature is especially prevalent in various applications such as healthcare monitoring, smart cities, and industrial IoT, where it leads to variable data rates and patterns, causing imbalanced datasets that challenge data processing and machine learning models.
— **Temporal and Spatial Differences**
The geographically distributed IoT components and temporal factors affect data collection, leading to imbalances. For instance, variations in the time of day or season.
— **Limited Resources**
The processing of selected data due to limited computational resources causes the data to be imbalanced.
— **Specific Transmission and Filtration**
To preserve bandwidth and storage, the Fog nodes broadcast and filter data selectively, resulting in an unbalanced dataset. Environmental monitoring devices, for instance, might only send data when values deviate from expected ranges.
— **Delay in Data Processing**
Temporal imbalances in data availability caused by variations in processing latencies might impact real-time analytics. For instance, outdated data utilized in decision-making may arise from a delay in data processing caused by computational overload.
— **Loss of data**
Data imbalance can result from gaps in datasets caused by data loss during transmission. For instance, the loss of vital patient data may result in an underrepresentation of specific medical conditions.
— **User interactions**
Data imbalance is introduced by the way consumers interact with IoT components changing over time. In particular, IoT applications focused on customers.
— **Environmental Factors**
External conditions have the potential to impact the data generated by Internet of Things components, resulting in data imbalances caused by situational or seasonal causes.
— **Advancement in Capabilities of IoT Components**
With the advancement of IoT technology, newer IoT devices generate more frequent data than older devices, leading to data imbalance. For example, the machinery upgraded with more sensors generate more data.
— **Data Quality**
Imbalanced data is the result of changes in data quality, such as errors, missing numbers, and noise. For instance, during harsh weather, sensors in weather monitoring systems that rely on outdoor sensors may malfunction and provide distorted data, which would reduce the reliability of the dataset as a whole.

— **Limitations on Energy**
Energy limitations may cause selective data transmission, which could provide data that is imbalanced. Trackers may lower data transmission frequency in remote wildlife tracking applications to save energy, which could result in gaps in the tracking data.
— **Growing Data**
The availability of historical and present data may become imbalanced over time as newer data replaces or becomes less relevant for previous data. For instance, data loss may occur when outdated client information is archived to make way for more recent information.

1.7 Motivation and Goal of the Article

The inspiration for the article and its objective are as follows: A few research studies have been conducted on handling imbalanced data in Fog computing, but it is still in its infancy. The previous studies provided a basis for the Fog computing architecture and a brief overview of different techniques to handle binary-class and multi-class imbalanced data for batch and stream data processing problems. However, as Fog computing devices are resource-constrained to handle imbalanced data, a lightweight technique is required for multi-class imbalanced data stream problems. This analysis has led us to an open issue for driving multi-class imbalanced data streams in the Fog. As noisy and incomplete IoT streams can create uncertainty, there is a need to define a mechanism for resource-constrained devices at the edge to handle imbalanced stream data that continuously updates instances and predicts novel and recurring classes that appear after a long time. So, it is necessary to thoroughly assess the literature on these imbalanced data handling techniques for batch and stream data. It is also essential to describe the architecture of Fog computing and its unresolved challenges, particularly in handling multi-class imbalanced stream data in Fog computing. We give a complete evaluation, covering all the paths to connect these holes. The foundation of Fog computing, numerous imbalanced data handling techniques, and a full assessment of the approaches used up to now for handling imbalanced data in Fog computing are all presented in this complete survey, which focuses on multi-class dynamic imbalanced stream data.

1.8 Contributions

The following are the major contributions of the present study:

— The study classifies and thoroughly examines the existing imbalanced data handling techniques, concentrating on imbalanced multi-class stream data handling techniques based on sampling, algorithmic, cost-sensitive, and ensemble approaches and examining their strengths and weaknesses.
— It gives a thorough explanation of the foundation of imbalanced data and its various forms, including batch (binary-class and multi-class) and stream (binary-class and multi-class) imbalanced data. Moreover, it delves into a comprehensive evaluation of the Fog computing paradigm for imbalanced data.
— This research describes the various performance metrics used in the literature. The metrics used for the evaluation of existing imbalanced data handling techniques are categorized into both binary-class (Accuracy, Kappa, MCC, Precision, Recall/Sensitivity, Specificity, F1-measure, G-measure, G-mean, and AUC) and multi-class (AveAcc, Average Precision, Mean Accuracy, Mean F-measure, MAUC, Kappa and Probabilistic AUC) metrics. For binary-class data, accuracy was thought to be the most popular metric, whereas MAUC was thought to be popular for multi-class data.
— In a non-stationary environment, concept drift occurs when the data and target concept evolves over time. When it coexists with class imbalanced, it affects predictive performance, and only a few approaches address this problem. In this survey, concept drift identification

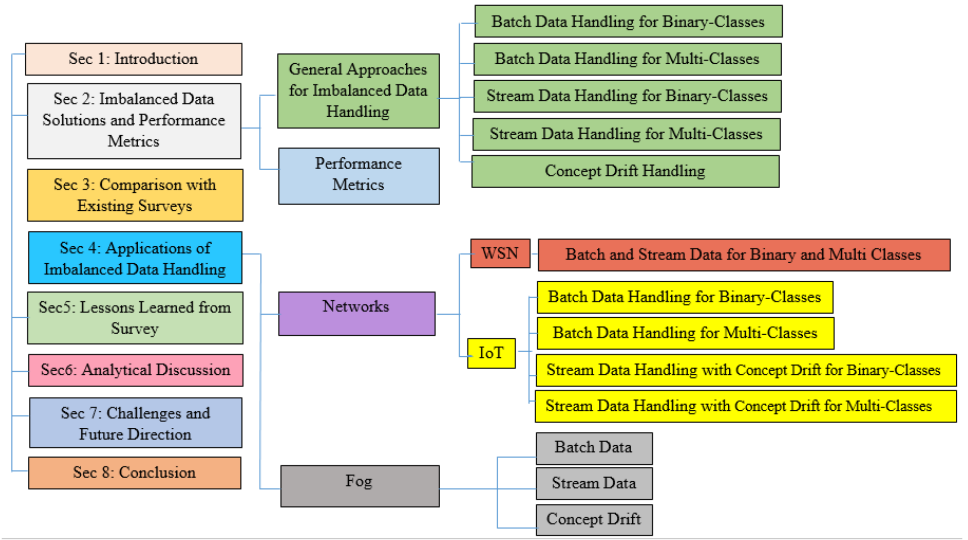

Fig. 3. Organization of the article.

in imbalanced stream data in different networks is thoroughly examined for the first time, focusing on the Fog network.

— The study also shows a research gap in the area of multi-class imbalanced data streams with concept drifts in Fog computing, which has to be filled.

1.9 Article Structure

The article structure, as described, outlines the organization of the research content, starting with the discussion of solutions for handling imbalanced data are detailed in Section 2, specifically in Section 2.1 and Section 2.2 gives the performance metrics to evaluate the effectiveness of various solutions. Section 3 elaborates on a comparison of existing surveys. The applications of imbalanced data handling techniques is presented in Section 4, which is further divided into subsections. WSN and IoT network methodologies for dealing with binary and multi-class imbalanced batch and stream data and concept drift handling are covered in Section 4.1 and Fog computing in Section 4.2. Section 5 presents the analytical discussion about the surveyed techniques, and the lessons learned from this survey report are presented in Section 6. In Section 7, we highlight the challenges and refer to future visionary research. Section 8 concludes the survey. Figure 3 displays the structure of the article.

1.10 Methodology

This comprehensive survey report implements PRISMA system to ensure multi-class imbalanced data and concept drift while maintaining transparency, consistency, and repeatability in the screening stage of the review procedure. The following sequential steps are used to describe the methodology. Integrating the PRISMA structure with the article's organization enhances the systematic and rigorous approach to reviewing and presenting the findings on imbalanced data handling in Fog computing.

— **Identification**

A comprehensive search study was designed to find the most important papers on the prediction of multi class imbalanced data with concept drift. Academic databases, Google Scholar,

IEEE Xplore, ACM Digital Library, ScienceDirect were searched using key terms such as "IoT," "cloud," "imbalanced data," "concept drift," "Fog computing." This yielded 150 records from Google Scholar, which were added to reference software after duplicate removal.

— **Screening**
Following the removal of duplicates, 130 distinct files were left for more review. Then, for finding the relevance of each document, the title and abstract of each document were studied and irrelevant documents were excluded in this phase. A more precise list of studies that are eligible is produced by the screening process.

— **Eligibility and Exclusion**
After screening, 130 papers were reviewed. Upon full-text review, 27 were excluded, leaving 103 articles that met the requirements.

— **Inclusion**
Finally, 103 studies met the predefined parameters and were included in the systematic review on Multi-class imbalanced data handling in Fog computing with concept drift.

2 Imbalanced Data Solutions and Performance Metrics

This section presents solutions for addressing imbalanced data and discusses relevant performance metrics to evaluate model effectiveness.

2.1 General Approaches to Handle Imbalanced Data

This section focuses on exploring fundamental strategies for addressing imbalanced datasets. It delves into both data-level and algorithm-level solutions that play a crucial role in mitigating the challenges posed by imbalanced data in various domains.

— **Data-level Solutions:** To tackle the challenge of imbalanced data, a sampling procedure can be employed. It is a preprocessing technique. By repeating the observations, minority class instances are multiplied in oversampling. In contrast, majority class instances significantly decrease in undersampling to maintain an equal number of occurrences in two different classes. In hybrid sampling, both sampling techniques are combined. Several ideas emerged under these categories, with non-heuristic preprocessing techniques such as random undersampling and random oversampling being the simplest.

— **Algorithm-level Solutions:** It is an alternative solution for data preprocessing to deal with imbalanced data. It is a classifier training procedure instead of modifying the training set. The imbalance in the training data samples can be corrected through the weighted distance function without affecting the class distribution [11]. The algorithms that are used for handling the imbalanced data are SVM bias, Naïve Bayes, and Neural Network.

— **Ensemble-level Solutions:** Ensemble learning and ensemble classification rely on several classifiers' votes to evaluate the actual class label of samples. This procedure builds different classifiers, each focusing on a unique set of characteristics or examples. The diversity of the training sets of classifiers causes the system to be varied. This heterogeneity between classifiers develops an ensemble-based system and helps to increase its robustness against noise. Because none of the classifiers uses the entire dataset, therefore, it performs better on data that has not been previously seen.

— **Cost-sensitive-level Solutions:** This approach generates classification algorithms for each class with a different misclassification cost. It necessitates understanding the cost of misclassification, which varies with every dataset and is sometimes not able to be known or challenging to compute. Furthermore, the algorithms must compute the misclassification cost for each class or instance while optimizing. There are two primary sub-categories of cost-sensitive learning algorithms: The first sub-category directly incorporates the

Table 1. General Approaches for Imbalanced Data Handling

Category	Strength	Weakness
Resampling	— Balance class distribution through resampling — Applicability to any learning algorithm [12] — Modify original training dataset — External approach [13] — Independent from the classifier	— Overfitting [14] — Lose important information — High learning time [15]
Algorithmic	— Provide good prediction accuracies [16] — Modify existing classification method — Internal approach	— It takes into account the error rate rather than data distribution — Fixed to the pre-determined learning algorithm — Cost-sensitive towards minority class
Ensemble-level	— Overcome imbalanced by forming sub-samples — Overcome computational load — Prevent performance degradation	— The more under-fit/over-fit models among the total ensemble models, the more adversely it affects the well-learned classifier
Cost-sensitive	— Focus on different misclassification costs of classifiers for different classes	— Sensitive to noise and outliers

cost of misclassification into the training procedure. The second sub-category is called meta-learning and it modifies the outputs of the classifier or the training data but not the training process. Meta-learning-based solutions can be used in two separate stages of the classification process, for example, preprocessing and postprocessing.

Table 1 summarizes the strength and the weaknesses of various imbalance data handling techniques.

Numerous real-world applications are more concerned about the categorization of imbalanced datasets. Binary-classification problems, where one class greatly outnumbers the second, have received the majority of attention in the literature on imbalanced classification. In addition to that, skewed class distributions can also cause multi-class difficulties that involve more classes, and one of them contains more instances than all other classes. We have grouped general approaches to addressing imbalanced data in this section. These techniques are further divided into stream and batch data, having both binary and multi-classes, as mentioned below:

2.1.1 Batch Data Handling for Binary Classes. As far as binary-class data processing is concerned, both batch data processing and real-time processing are included. Batch processing requires processing a significant volume of previously stored data, whereas real-time processing entails processing stream data. Stream processing consists of an infinite number of tiny batches. In the case of batch processing, the data faces a few problems. One of them is the data-imbalance problem. Various approaches have been used to overcome this problem in binary-class datasets. A few of them are given below:

Table 2. Comparison of Data-level Solutions for Imbalanced Data Handling Techniques

Technique	R-Type	Acc	G-mean	F1-m	A	R	P	Cov	MI	Co-C	MSD	Bal	DS	Ref
Heuristic oversampling based on K-mean and SMOTE	OS	×	✓	✓	✓	×	×	×	×	×	×	×	UCI, KEEL, MEDELON	[17]
KMFOS	OS	×	×	×	×	✓	×	×	×	×	×	×	Project from NASA, softlab	[18]
MTDF	OS	✓	×	×	×	✓	✓	✓	✓	×	×	×	"LINKS OF COMPARING OVERSAMPLING"	-
Empirical comparison	OS	×	✓	✓	✓	×	×	×	×	×	×	×	Keel	[19]
Comparison of oversampling and undersampling	OS	✓	✓	×	×	×	×	×	×	✓	×	×	7-year freshmen students data	[20]
ECO-ensemble framework	OS	×	×	×	×	×	×	×	×	×	✓	×	UCI	[21]
SMOTE variants - A comparison	OS	×	×	×	×	×	×	×	×	×	×	×	Libras	[22]

✓ =metrics focused in technique; ×=metrics ignored; R–Type=Resampling Type; OS=Oversampling; US=Undersampling; Acc=Accuracy; F1-m=F1-measure; P=Precision; Cov=Coverage; MI=Mutual Information; Co-C=Correlation Coefficient; MSD=Mean standard deviation; Bal=Balance; DS=Datasets; Ref = Reference.

— **Resampling**

Class-imbalanced datasets are prevalent in different domains, including health, security, banking, and others. A typical supervised learning algorithm tends to be biased towards the majority class when dealing with imbalanced datasets. The solution proposed to solve the class-imbalance problem is data resampling. The data-level solution formally known as resampling provides a means to modify data distribution and yields a revised set with balanced data distribution.

(1) **Oversampling**

Even though the accuracy is good, the correct specification rate for the minority class suffers in an imbalanced dataset situation. To remedy the problem, the oversampling approach was applied without regard to the loss of accuracy. Furthermore, an arbitrary oversampling strategy may result in bias. Oversampling tactics were proposed by many researchers in various formats, some of which are listed below:

— Reference [17] coupled the k-means clustering method with SMOTE to produce higher classification results than training with unmodified, imbalanced data. This technique solved both the between-class and within-class imbalances by inflating scarce minority regions.

— Reference [18] provided a cluster-based oversampling with noise filtering (KMFOS) approach for handling the problem of class imbalanced **Software Defect Prediction (SDP)**. KMFOS first divided faulty instances into K clusters and then interpolated between instances of each of the two clusters to generate new defective examples. The researcher then improved this cluster-based oversampling with the **Closest List Noise Identification (CLNI)** to clear the noise occurrences. In Table 2, the tick marks (✓) indicate the intended criteria, while the crosses (×) show the metrics ignored by the researchers.

— **Undersampling**

Different researchers presented undersampling strategies in various forms, some of which are given below:

— Many academics have suggested informative undersampling procedures to prevent the loss of useful information. Unlike K-specific clusters, the cluster-based undersampling strategy based on distance-based instance concepts proved beneficial for dataspace that was highly clusterable.

Table 3. Approaches for Undersampling and Hybrid Sampling

Technique	Dataset	Tool	Parameters	Ref
Boosting-driven cluster-based undersampling (undersampling + clustering)	Breast cancer, Diabetes, German credit card, Ionosphere, Blood transfusion, Spambase	MATLAB	Recall, Precision, G-mean, F1-measure, Specificity	[15]
Clustering-based undersampling in class-imbalanced data (undersampling + clustering)	44 small-scale datasets used by Glar et al. as well as two large-scale datasets, namely, the breast cancer and protein homology prediction	–	Accuracy	[14]
Neighborhood-based undersampling framework	66 datasets from UCI and KEEL	–	Sensitivity, G-mean, Precision, F1-score	[12]
TPHM	Uremia dataset	–	Accuracy	[24]
SUNDO	A synthetic dataset of 134 samples and real-world datasets from the metal industry	–	Accuracy	[13]
Undersampling + ensemble	Dataset from steel manufacturing plant	Python	G-mean, F1-score, Recall, Precision	[25]

— By combining the undersampling of majority occurrences with classifier learning, an adaptive undersampling strategy was suggested in Reference [15].
— Undersampling was done iteratively inside an ensemble learning framework that is used to control the training flow for future iterations. The AdaBoost ensemble model was used for the classifier training along with the decision tree C4.5 as the weak learner [23].
— Reference [14] introduced two undersampling strategies. The first strategy uses the cluster centers to represent the majority class, whereas the second strategy uses the nearest neighbors of the cluster centers. It can reduce the risk of removing useful data from the majority class.
— Neighborhood-based undersampling framework [12] identified and eliminated majority class instances from the overlapping region. First, it maximizes the visibility of minority class instances. Second, it prevents excessive eliminations and minimizes information loss.
— Reference [24] proposed a hybrid imbalanced-class decision-tree rough set model to integrate the knowledge of experts. The accuracy of the hybrid sampling and oversampling methods was very close.
— In Reference [13], a new resampling method was presented, combining an oversampling and an undersampling technique. It outperformed the widely adopted combination of SMOTE oversampling and random undersampling.
— The researcher suggested an ensemble learning-based undersampling technique using **Extreme Gradient Boosting (XGBoost)** and SVM [25]. For producing the training set for this ensemble method, the patterns were generated randomly after sampling on the majority set. This methodology helps in improving the classification tasks. Table 3 summarizes the information.

— **Ensemble**
Multiple classifier systems, also known as ensemble-based classifiers, have been shown to improve a single classifier's performance by integrating various base classifiers that collectively perform better than each one used alone. Classifier ensembles have become more common as a solution to the class-imbalance problem. Probably, 218 publications out of the 527 reviewed papers in a survey report [26] presented new ensemble models to address real-world problems.

Table 4. Ensemble-level Approaches for Data Handling

Technique	Dataset	Tool	Parameters	Ref
BalancedBoost	UNIBS dataset	−C4.5 as base classifier	Precision, Recall, F-measure, Accuracy,	[27]
Random Balance	HDDT, KEEL	Weka (J48 as base classifier)	AUC, f-measure	[28]
Gradient Boosting	Ivshina, Wang, Sotiriou, EM	−CART as base	G-mean, AUC	[29]
EPRENNID	35 datasets from UCI and KEEL	−1NN	AUC, G-mean	[30]
SwitchingNED	33 datasets from UCI and KEEL	Decision Tree C4.5	Average AUC	[31]

In "BalancedBoost," proposed by Reference [27], RUSBoost was modified and resampling took place using AdaBoost.M2 algorithm weight. In another technique, given by Reference [28], an amalgam of RUSBoost and SMOTEBoost was called "RandomBalance Boosting." The Adaboost.M2 method was paired with random balanced sampling to produce an ensemble capable of handling imbalanced classes. In Reference [29], the optimization of an arbitrary differentiable loss function was allowed by the gradient-boosted trees. An ensemble approach proposed by Reference [30] for classifiers specifically focused on data preprocessing was called **EPRENNID (Evolutionary Prototype Reduction based Ensemble for Nearest Neighbor Classification of Imbalanced Data)**. The hybridization of prototype selection and prototype generation for ensemble building results in the distinct reference sets of a K-NN. Both systems were created using evolutionary algorithms, and both methods adjust for imbalanced class, primarily done by taking relevant performance measurements into the fitness function. According to Reference [31], the **undersampling Switching Nearest Enemy Distance** was known as **USwitchingNED**. It randomly swaps the labels of instances of the majority class to achieve diversity. Table 4 outlines a few ensemble-level solution approaches for imbalanced data handling.

— **Cost-sensitive**

As opposed to the resampling strategy, cost-sensitive learning is more computationally effective, making it a better choice for Big data streams. It is far less common than resampling methods, as evidenced by the survey report that found just 39 of the examined papers employed it [26].

Reference [32] improved classification accuracy along with the consideration of variable misclassifcation cost. The approach presented by Reference [33] automatically learned the representations of features for both underrepresentative (minority) and overrepresentative (majority) classes. Reference [34] directly incorporated a cost-sensitive function into the classification paradigm and employed differentiable evolution for the optimization of the cost matrix. The research proposed by Reference [35] used an adaptive differential evolution to tackle optimizing the misclassification cost. It was an effective solution to tackle unknown misclassification costs. Reference [36] has combined the cost-sensitive method with a threshold strategy to increase the accuracy of the minority class, and for this purpose, it used a cost-sensitive factor for assigning larger weights to the underrepresentative (minority) classes and punishing the overrepresentative (majority) classes. A few of the works on this strategy are given in Table 5.

— **Algorithmic**

Traditionally, classification algorithms have been unable to deal with the problems of imbalanced data, since they are biased against the dominant class. As a result, algorithms have become unable to classify the most demanding minority class.

The proposed model of Reference [37] entailed developing an approach to genetic programming that employed hierarchical linguistic variables. It suggested combining SMOTE with

Table 5. Cost-sensitive-level Approaches for Data Handling

Technique	Dataset	Tool	Parameters	Ref
AdaC-TANBN	Heart, ILPD, Dermatology, CCRF	R3.5 mathematical development environment	Accc, Sensitivity, Specificity, AUC, ROC	[32]
CoSen CNN	MINST, CIFAR-100, Caltech-101, MIT-67, DIL, MLC	–	F-measure, G-mean	[33]
CSDBN-DE	42 datasets from KEEL	–	Accuracy, ErrorRate	[34]
ECS-DBN	58 datasets from KEEL	Python	Acc, G-mean, AUC, Precision, F1-score	[35]
CSHCIC	protein dataset (DD, F194), SUN	–	ACC, F1 Hierarchical measure	[36]

Table 6. Algorithmic-level Approaches for Data Handling

Technique	Dataset	Tools	Parameters	Ref
HFRBCS	44 datasets from KEEL	–	–	[37]
Compact evolutionary IVFRBCS	BI, BC, WSI, FESI, DT, AL, SL, ARB, FD, Len, LA	–	G-mean	[16]
Fuzzy KNN	Ionosphere and New-Thyroid from UCI, Wisconsiin, Phoneme, Vehicle0, and Glass2 from KEEL	–	Precision, Recall, F-measure, AUC, G-mean	[38]
Improved Fuzzy KNN	Ionosphere, Pima, Transfusion, Spectfheart, Wine quality from UCI, Phoneme, Vehicle0 and Ecoli1, Yeast-2-vs-4, Ecoli4	–	F-measure, AUC, G-mean	[39]
Industrial IoT (IIoT) testbed	Real built dataset	–	Accuracy, False Alarm Rate, Undetected rate, Sensitivity, MCC	[40]

algorithmic alterations, such as using a hierarchical knowledge base. For the purpose of balancing the weights of the fuzzy rules that are linked with different classes, Reference [16] employed a rescaling mechanism. In the technique of Fuzzy KNN [38], the benefits of the neighbor-weighted KNN approach were merged with fuzzy logic. Its results were better than NWKNN and Adpt-NWKNN. An improved Fuzzy KNN given by Reference [39] was an adaptive K-nearest neighbor strategy to handle the imbalance problems. Besides, for the purpose of getting the test instance memberships from imbalanced data, it was joined to fuzzy K-nearest neighbors. The fuzzy memberships of data instances using adaptive KNN were more accurate than simple fuzzy KNN. Another study, Reference [40], described a testbed and created an **intrusion detection system (IDS)** that is based on machine learning. Table 6 outlines a few algorithmic-level solution approaches for imbalanced data handling.

2.1.2 Batch Data Handling for Multi-classes. In recent years, the researchers spent much effort on the situations of data imbalanced in binary-class, which has only two classes. Various real-world applications are suffering from multi-class imbalanced classification issues due to the widely disparate distribution of data classes. It is frequently employed in numerous fields, including text categorization, human activity detection, and medical diagnosis. Learning from many classes makes data-mining techniques more challenging when considering overlapping across classes [41], a dearth of representative data, and mixed types of data [42]. Unfortunately, applying the solutions that are suggested for the binary-class problems to multi-class imbalanced issues can be invalid,

and some techniques are impossible to be applied directly to the imbalanced situations of multi-classes [43]. It is proposed that, to deal with the classification of multi-class problems, decomposition strategies are preferably used. Binary-class imbalanced data techniques have generated more interest in the research community. These techniques allow you to break down multi-class problems into smaller sub-problems of the binary-classes that can be easily solved. This section gives two of the most commonly used binary decomposition techniques:

(1) **One-vs.-One**
   — The concentration on one-vs.-one does not affect the positive and negative class distributions.
   — It reduces the computational time.
   — The decision boundaries of each binary-class problem could be easier to determine than the "one-vs.-all" transformation.
(2) **One-vs.-All**
   — Simpler
   — Not reliable, because when samples from classes that are not "small" enough are crowded into one class, the distribution becomes extremely imbalanced, especially if the surviving class is minor [44].

Various approaches have been developed for addressing the major problem of multi-class imbalanced class distribution. These strategies can be classified into four levels: data level, algorithmic, cost-sensitive, and ensemble level. Table 7 summarizes general approaches for handling multi-class imbalanced data.

2.1.3 Stream Data Handling for Binary Classes. The IoT components contain sensors of various types that collect or generate various data throughout time for numerous sectors and applications in the **Internet of Things (IoT)** era. These IoT components can produce massive or quick (real-time) data streams while relying on the nature of the application. The data from IoT components can be constantly gathered or transmitted to create a huge data source. Data created or retrieved in a brief time interval is called "streaming data." It works to gain quick understanding and/or to make rapid decisions. "Big data" includes large datasets that are too large for traditional technology and software platforms to store, manage, process, or analyze. Because their needs for an analytic response are not the same, these two techniques should be considered differently. Big data analytics insights can be supplied within a few days of data collection, but the analytics of streaming data insights must be available immediately.

Applying analytics to these data streams to extract new knowledge, foresee future disclosures, and make judgments in real time is essential. It identifies the IoT as a technology that enhances the quality of life. Large-scale streaming data, heterogeneity, time and spatial correlation, and high noise are properties of IoT data that set it apart from ordinary big data.

2.1.4 Stream Data Handling for Multi-classes. In the case of stream data, new data samples are continuously created, and their properties evolve when they exist. On the contrary, when the issue appears, it becomes non-stationary. Therefore, the classifiers must exhibit great speed, low computing cost, accuracy, and the ability to accommodate new examples continuously. Some data stream applications are more class-imbalanced, i.e., one of the classes is underrepresented. This causes great learning difficulties, because traditional machine learners ignore or overfit the minority class. As an **imbalanced ratio (IR)** evolves, a fixedIR cannot be used, the problem may become balanced, classes may switch roles, and overlapping with other classes are a few such difficulties. Multi-class imbalanced learning suffers from more difficulties than two-class problems even in the case of offline learning.

Table 7. General Approaches for Multi-class Imbalanced Data Handling

Technique	Method	Parameters	Highlights	Ref
Oversampling and Undersampling	It combined the Neighbor Cleaning Rule (NCL) to remove the outliers with the SMOTE to increase the samples.	Recall	When compared to individual procedures, the recall rate is higher.	[45]
FH-GBML based on OVO	In the first step, one-vs.-one binarization and in the second step, the SMOTE algorithm was applied to again balance the data before the process of pairwise learning.	Probabilistic AUC	It outperformed the basic and paired learning multi-classifier approaches.	[41]
Ensemble	Binary ensemble learning methodologies were used to support the one-to-one scheme. Then, the results were combined using the vote aggregation strategy to recreate the original multi-class challenge.	AvgAcc	It demonstrated how well decomposition techniques and ensemble learning interact.	[43]
AMDO	GSVD (Generalized Singular Value Decomposition) is introduced for the mixed-type of data by AMDO, which partially develops the strategy of balanced resampling and also optimizes the sample synthesis.	P-min, P-avg, AUCm	It hinders performance when dealing with low-dimensional datasets.	[42]
SMOTE	Initially, K closest neighbors from the minority class are chosen, and their difference is computed. Then, the fresh samples are created within the range of differences.	AUC	The class covariance structure is not preserved. Overlaps between classes and messes up class boundaries.	[46]
Oversampling	The oversampling approach is dependent on the joint probability distribution of data attributes.	-	Enhances accuracy while preserving covariance structure.	[47]
MDO	This distance-based oversampling considers the class with the most samples as majority, while the remaining classes becomes minorities. Additional samples for each minority class are generated in proportion to the number of examples in the majority class.	MAUC	Effective in a multi-class imbalanced situation with overlapping classes. The structure of class co-variance is preserved.	[48]
Clustering-based undersampling	This technique is used to increase the classification accuracy of the class with a smaller number of instances.	MI's F-measure	Clustering-based undersampling produces better results than other undersampling.	[49]
Oversampling and undersampling	Improved SMOTE (ISMOTE) as an oversampling technique is paired with distance-based undersampling (DUS).	AUC and G-mean	It produced better outcomes than oversampling or undersampling.	[50]
Spectral clustering	OVO decomposition is applied, followed by spectral clustering to separate minority classes into subspaces, which are then oversampled based on data features	P-min, P-avg, MAUC	It shows best performance in comparison to multi-class imbalanced learning.	[44]
Feature extraction with random sampling	New features are extracted using multi-intra clusters to control redundancy in multi-class imbalanced classification, selecting features with highest similarity. Then, a resampling technique is applied.	MFM, MAUC, MAcc	The highest average of MAcc, MFM, and MAUC shows the potential of this method.	[51]
Self-inspected adaptive SMOTE (SASMOTE)	The "visible" nearest neighbors are found using the nearest neighbor algorithm, which produces samples that are likely to belong to the minority class. The produced samples that are extremely ambiguous and inseparable from the majority class are then separated using a self-inspection technique for uncertainty elimination.	F1 score	Recommended when there are a lot of nearby neighbors and optimal average performance requires fine-tuning the uncertainty score threshold.	[52]
The dynamic sampling	All samples are fed into the deep neural network for the current iteration, and the performance metrics are calculated for the neural network	AUC	Deep learning algorithm outperformed the other algorithms that were chosen	[53]

In the offline version of data, the classifier detects minority and majority classes before the learning begins. However, online learning has two important characteristics. The first is the underrepresentation of minorities in class samples, and the second is the incremental arrival of samples in the learner. This can cause a few of these problems.

— First, it is impossible to determine the minority class ahead of time, because the learner lacks a comprehensive picture of the data.
— Second, the status of the minority class can alter over time. Since there are fewer samples from minority classes than from majority classes, updating the learner with correctly categorized examples may encourage overfitting toward samples from the majority class.

Therefore, the work presented in Reference [54] updated the base classifier after receiving each sample that the learner correctly categorized, instead of updating the learner. This work allowed its classifier to misclassify samples up to an acceptable level to avoid erroneous updates. However, it did not have any method for coping with concept drift and evolving class properties.

Class decomposition simplifies multi-class imbalanced data streams, but it causes a few problems when combining binary classifiers. The number of classes and the classifiers could evolve just like with data streams, so it becomes difficult to combine multiple binary classifiers. Moreover, binary classifiers are trained without full knowledge, which leads to classification ambiguity. The work of Reference [55] dealt with MOOB and MUOB, which processed multi-classes directly without using the decomposition of classes.

Further, some of the studies, like Reference [56], have focused on recurring classes. A class becomes a recurrent class when it returns from a prolonged absence from the stream. The technique that has been used in the study is CLAM, a class-based approach rather than a chunk-based approach, because a chunk-based approach keeps a fixed-size ensemble. In the chunk-based approach, when a class vanishes, all models developed with that class are discarded, and no model can recognize the class when it reappears. As opposed to recurring classes being mistakenly identified as novel classes, the CLAM technique discovers novel classes. It eventually increases the accuracy of the classifiers.

To address the emergence and disappearance of concepts in a data stream, the work of Reference [57] offered a method that employed continuous and active learning. AnyNovel detected both normal (driving) and abnormal (sudden fall) novel concepts. AnyNovel has the ability to adapt to changes by recognizing recurring novel concepts as well as abandoned (forgetting) concepts.

2.1.5 Concept Drift Handling. A stream of data is a constant flow of data that arrives at a high rate. In a dynamic streaming environment, the data continuously changes over time along with the evolution of the stream. The changing nature of data results in the emergence of a few unique characteristics, one of which is the concept drift that occurs with the continuous change in the concept of the data. These innovative concepts could be examples of fraud detection, network intrusion detection, or sudden drop detection. It would be an innovative concept that the system has never heard of or been taught about.

The stream data is categorized into three types [58]. The first type of classification technique is based on a single model. It updates the single classification model incrementally, and it responds to drift effectively. The second type of classification technique is an ensemble-based technique, which maintains a number of classification models. Some new classification models are gradually replacing the old ones in this category. And in the third type (hybrid), single and ensemble approaches are combined.

When the statistical features of data in a dynamic stream environment change at different time intervals, the problem of concept drift arises. This concept drift can be virtual as well as real. Most often, it is characterized on the following basis [59]:

First, it can be categorized on the basis of speed of drift, where it can be abrupt, in which changes suddenly occur from one concept to another, or gradual, in which transformations happen gradually over time. Second characteristic is the severity of drift, which can be local as well as global. Third is the reccurency in which the concept drift can be seen in two ways: Either it can be a new concept (Novel Concept) or an old concept (Recurrent Concept). The concept drift may introduce many significant challenges for **machine learning (ML)** models. For example, the change in class labeling in various timesteps may decrease accuracy. This problem arises in the context of online learning, where patterns shift over time. As a result, machine learning models must react quickly to changes to preserve the accuracy of their findings. The machine learning model learns in two modes [60], retraining as well as incremental. In the case of retraining, the model is trained on the first batch of data, but once drift is detected, the old model is rejected and the newly predicted model is developed; this is then applied to each new instance of data. On the contrary, the incremental learning works by updating the predicted model regularly.

Another challenge to dealing with drift is the recurrence and adjustment of the new concept. Drift recurrence is more difficult than the novel concept, because it is more challenging to keep track of previous concepts. The buyer's purchase behavior to buy the items is a good illustration of recurrent drift. For example, every summer, the whole activity of buying clothes is repeated. The following fundamental approaches are used to deal with concept drifts [61]:

— The first is instance selection, which aims to identify instances relevant to present concepts.
— The second technique is instance weighting, which uses the ability of learning algorithms to interpret the weighted instance.
— The third method is ensemble learning, which keeps track of a series of concept descriptions, the predictions of which are combined by using a voting system, or a highly relevant description is chosen. Finally, the activity of combining the base classifiers is performed through static (voting, weighted-voting, CVM) or dynamic (DS, DV, DVS) techniques.

When the environment is non-stationary, the distribution of classes is mostly imbalanced. The other problem in this imbalanced data stream is concept drift, where the target class keeps drifting all the time. The work performed by Reference [62] accommodated the inclusion of a small number of minority cases that had previously been approved in the training phase. In accordance with the current majority collection size, the number of acceptable prior minority cases increases. The Mahalanobis distance was used to determine the priority level of acceptance. This algorithm improved the prediction accuracy for the minority class. This work was not strictly incremental and was suitable when earlier observed data was kept and later used.

An **Online-MC-Queue (OMCQ)** algorithm that learns multi-class imbalanced setting was proposed by Reference [63]. It utilized a queue-based resampling method that created an instance queue for each class. This algorithm was able to dynamically adapt to changes using DDM algorithm while simultaneously dealing with multi-class imbalanced data.

A systematic study [64] dealing with class imbalance and concept drift is presented. A summary of several approaches was provided in Table 8, including DDM-OCI, LFR, PAUC-PH, RLSACP/ONN, ESOS-ELM, OOB/UOB using CID. These approaches were not applied to multiple classes. According to this study, the performances of RLSACP and ESOS-ELM were not good. LFR and DDM-OCI were sensitive to concept drift. To detect change, the researcher employed an adaptive class imbalance technique (OOB). The best strategy overall was determined to be the combination of PAUC-PH and OOB based on the observations made regarding minority-class recall and G-mean. Researchers have recently focused a lot of attention on this issue, because many learning problems need to be resolved. To achieve that, this study comprises some open

Table 8. Algorithms Handling Concept Drift and Class Imbalance Problem

Algorithm	Detection	Advantage	Limitation
Linear Four Rates (LFR)	Concept Drift	The detection of data change over time	High rate of false discovery compared to hybrid concept drift
PAUC	Concept Drift	Fast concept drift detection	The time dependence between instances is not taken into account
RLSACP	Concept drift and imbalanced data distribution	Detecting concept drift over imbalanced data classes	Inaccurate for datasets that are nonlinear and/or nonseparable
MWMOTE	Distribution of imbalanced data	Solving multi-class issue	For certain kinds of datasets, oversampling is insufficient
WOS-ELM	Distribution of imbalanced data	Not required to keep previously acquired information	It is assumed that the classes do not change with the passage of time

challenges and an experimental investigation. The development of a more efficient technique to detect concept drift in imbalanced data streams is one of them.

Another review of a combined problem of concept drift with class imbalance has been presented by Reference [65]. This work gave a comparative study of different classifiers on the class imbalance dataset with concept drift. Single learner and ensemble classifiers were used in this study and tested on a variety of datasets, including real-world datasets and synthetic data streams like SEA, electrical, and KDD datasets. It was observed that class distribution had a high impact on the classification process. It was also noted that an ensemble-based algorithm provided better results when compared with a single classifier when dealing with concept drift. In the future, deep learning approaches can be used to deal with concept drift in class-imbalanced data streams. This work presented a few algorithms and their limitations used for concept drift with class imbalance issues.

Table 9 summarizes various approaches for concept drift handling.

Reference [77] presented two major ensemble-based techniques for the detection of concept drift from imbalanced data. SMOTE and Learn++.NSE were used together as the first technique. In the second technique, a sub-ensemble took the place of SMOTE and Learn++.NSE. Moreover, the algorithm was compelled to balance accuracy across all classes because of its class-independent error weighting scheme and penalty restrictions. This work proved that Learn++.NSE should be used for concept drift in data for the balanced classes. Learn++.NIE is a preferred algorithm in a situation where both majority and minority classes and concept drift require strong balance performance. By setting the ensemble size, it may be created considerably more quickly. Learn++.NIE gains knowledge from new data without needing access to data that has already been observed. For the proposal of a general framework of concept drift data streams with imbalanced data distribution, Reference [78] presented a new method for mining data streams that involves generating trustworthy posterior probabilities with an ensemble of models to fit the distribution across negative undersamples and positive repeated samples.

2.2 Performance Metrics

It is essential to employ appropriate performance metrics to evaluate the effectiveness of various solutions for handling imbalanced data in different domains such as Fog computing, **wireless sensor networks (WSNs)**, and IoT. The performance of learning algorithms on test data is commonly used to assess their quality. For this purpose, the predictions of the trained classifiers are compared to the true classes of the test data and various performance indicators are generated. We examine these metrics in both binary and multi-class issues.

2.2.1 Binary-class Metrics. There are three different scenarios depending on how we interpret the classifiers' output or the amount of information they supply: nominal class predictions,

Table 9. Approaches for Concept Drift Handling

Technique	Imbalanced	Key points	Datasets	Tools	Base classifiers	Parameters	Ref
RDDM	No	It removed the earlier instances of the concepts for detecting drifts and boosting final accuracy	48 artificial and 3 real-world	MOA	Naive Bayes	Accuracy	[66]
Comparative analysis	Yes	The finest concept drift detectors must detect all existing concept drifts closest to their right places	Artificial	MOA	Naive Bayes and Hoeffding Tree	Precision, Recall, MCC, Accuracy	[67]
Dynamic ensembles' integration	-	It outperforms the best stationary batch learning technique	Vitek-60 bacterial analyzer	Weka 3.4.2	Naive Bayes, C4.5 DT, KNN	Accuracy	[61]
Drift handling for prediction process	No	It shows the various alternatives for selecting training data for ML models that need to be retrained	-	Python	NB, NN, SVM, DT	Accuracy	[60]
Predict-Detect framework	Yes	It deals with adversarial drifts, e.g., data distribution changes, that alter the features of specific class samples	Synthetic, CAPTCHA, phishing and digits08	-	Linear SVM	Accuracy	[68]
Integrating Adadelta and Adamax	Yes	Momentum-based stochastic gradient descent techniques deals with concept drift passively	9 synthetic and 3 real	-	Hoeffding Adaptive Tree	Accuracy	[69]
ACNNELM	No	It provides improved accuracy, computing scalability, and concept drift adaptability	MINST and not-MINST	DeepLearn Toolbox	-	Accuracy, Cohen's Kappa	[70]
ISTM	No	ISTM changes the model after reading the intermediary data matrix again when new data arrives	CityPulse data	-	Linear regression	MSE accuracy	[71]
Comparative analysis	Yes	The combination of PAUC-PH and (OOB) was found to be the best out of all the other approaches tested for imbalanced data with concept drift	Artificial (SINE1 and SEA), Real-world dataset (Tweet, Weather, PAKDD)	-	Multilayer perceptron	Recall, G-mean	[64]
AUC estimation	YEs	EWAUCPH and GM-PH demonstrate a higher true detection rate than other concept drift detectors in the PH-test (TDR)	-	-	-	G-mean, EWAUC-PH, PMAUC, EWAUC	[72]
RBM-IM	Yes	It provides a taxonomy for the difficulties with multi-class data with novel concept drift	12 Real-world and 12 Artificial	MOA	Adaptive Cost-sensitive Perceptron	pmAUC and pmGM	[73]
HIDC	Yes	It uses resampling for imbalanced data and for concept drift weighting scheme replaces the worst classifier	City pulse weather dataset	-	-	Precision, Recall, G-mean and delay	[74]
DUE	Yes	It preserves limited classifiers, emphasizes misclassified samples, learns one chunk at a time, and manages various forms of drift	Synthetic and real datasets	MOA	VFDT	Recall, Precision, F-measure, G-mean	[75]
Imbalanced data analysis with drift	Yes	Local data properties and local drift were taken into account instead of global factors	Synthetic and real data streams	MOA	OOB, UOB, ESOS, VFDT, OB	G-mean, Recall	[76]
OMCQ	Yes	It functions independently of a base classifier, keeps queues for every class, and implicitly balances the data without requiring resampling	Cover type	Python, Scikit-Learn	Hoeffding Adaptive Tree, SAM, KNN	F-measure, G-mean, Cohen K statistic	[63]
Systematic study	Yes	A thorough review and experimental study for handling imbalanced data with concept drift	SINE1, SEA, Python, R, Java, scikit-learn, Weka, TensorFlow	-	-	G-mean	[64]

numerical scoring predictions, and probabilistic predictions. Now, we will look at each of these scenarios in terms of binary classes.

(1) Nominal Class Predictions: To assess the model, nominal class predictions compare the labels of the predicted class to the actual true class values. The confusion matrix is a cross-tabulation of actual and anticipated classes used to summarize how well classifiers perform. Depending upon the confusion matrix, many performance measurements may be constructed. A few of them are listed below.

— **Accuracy**

An accuracy measure is a type of performance metric that is commonly used to assess classification performance. It is the percentage of events that were correctly categorized. Accuracy and error rate calculations are widely used but they have a few limitations when dealing with imbalanced data. Low error rates or high accuracy can be easily achieved, and it is also assumed that errors are computed costly. In the confusion matrix, accuracy is represented by the diagonal elements and is calculated using Equation (1) and error using Equation (2) given below:

$$Acc = \frac{TP + TN}{N}, \tag{1}$$

$$Error = 1 - Acc. \tag{2}$$

— **Kappa**

The predicted accuracy is removed from the accuracy in the kappa metric. After that, $1 - Acc_e$ is used to normalize the value. The kappa value spans from -1 to 1, and values less than zero imply that the classifier performs worse than random guessing. The Equation (3) for Cohen's kappa is given below:

$$k = \frac{Acc_0 - Acc_e}{1 - Acce}. \tag{3}$$

— **Matthew's Correlation Coefficient (MCC)**

It is a metric that considers all confusion matrix values as well as mistakes and proper classification in both minority and majority classes. Equation (4) shows the MCC formula. MCC is a scale that spans from -1 to $+1$, with $+1$ reflecting the best possible forecast, 0 representing no better than chance, and -1 representing the worst possible prediction.

$$MCC = \frac{TP.TN + FP.FN}{\sqrt{POS.NEG.PPOS.PNEG}} \tag{4}$$

— **Precision**

The fraction of correctly categorized events among those labeled as positive is measured by precision. It is a metric for determining how accurate a model is. Its formula is given in Equation (5).

$$Precision = \frac{TP}{TP + FP} \tag{5}$$

— **Recall/Sensitivity**

The fraction of all positive events accurately labeled as positive is known as recall. The classifier's sensitivity to the positive/minority class determines how successful it is. Its formula is given in Equation (6).

$$Recall = \frac{TP}{TP + FN} \tag{6}$$

— **Specificity**
The classifier's efficacy on the negative/majority class is measured by specificity [79]. Its formula is given in Equation (7).

$$Specificity = \frac{TN}{TN + FP} \tag{7}$$

— **F-measure**
The F-measure employs a weighted harmonic mean of the positive predictive value and true positive rate also known as *accuracy* and *recall*. Its formula is given in Equation (8).

$$F - measure = 2.\frac{Precision.Recall}{Precision + Recall} \tag{8}$$

— **G-measure**
G-Measure is a variant of F-Measure that trades precision for recall by using the geometric mean rather than the harmonic mean. Equation (9) shows its formula.

$$G - measure = \sqrt{Precision.Recall} \tag{9}$$

— **G-mean**
It is another geometric mean-based measure that incorporates data from both minority and majority classes. Even if the negative instances are accurately identified, poor performance in predicting the positive cases will result in a low G-mean score. This metric is identical to conventional accuracy when the classes are evenly balanced. Equation (10) shows its formula.

$$G - mean = \sqrt{Sensitivity.Specificity} \tag{10}$$

(2) Numerical Scoring Predictions: To rank the instances, the methods use score-based ordering combined with predictions to award a grade to test samples based on how likely they are to belong to a certain class. The following is an example of numerical scoring predictions:
— **Receiver Operating Characteristic (ROC) Charts/Area under the curve (AUC):** The ROC curve determines both specificity and sensitivity for a variety of thresholds. Finding the ideal ratio of sensitivity to specificity can be done using the curve. The area under the ROC curve is called the AUC. An ideal model contains an area of 1, whereas the area of a worthless model is 0.5.

(3) Probabilistic Predictions: The numerical outputs linked with probabilistic predictions are examples of class probability. The Brier Score is commonly used to evaluate probabilistic scores. The fundamental idea is to compute the **mean squared error (MSE)**, with positive classes being represented as 1 and negative classes being coded as 0. This computation involves predicted probability scores and the real class indication. The Brier Score in its most popular form is shown in Equation (11).

$$BS = \frac{1}{N}\sum_{i=1}^{N}(p_i - o_i)^2 \tag{11}$$

2.2.2 Multi-class Metrics. The accuracy is helpful for binary dataset classification, but it does not provide a holistic view of how well our prediction model works. A few other metrics are required for the handling of multi-class imbalanced data.

— **AveAcc**
Each class is given equal weight by the average accuracy. The accuracy rate of each class is determined separately, and the average result is used for the final computation. The following is the formula for calculating the average accuracy given in Equation (12).

$$AveAcc = \frac{1}{m}\sum_{i=1}^{m} TRP_i \tag{12}$$

— **Average Precision**
It represents the overall accuracy of all classes, and its formula is given in Equation (13).

$$P_avg = \frac{1}{c}\sum_{i=1}^{c} P_i \tag{13}$$

— **Mean Accuracy (MAcc)**
The MAcc is calculated by averaging the accuracy rates of each class separately. The formula given in Equation (14) defines it.

$$MAcc = \frac{\sum_{i=1}^{n} MAcc_i}{r} \tag{14}$$

— **Mean F-Measure (MFM)**
It calculates the f-measure of each class and then uses the average to calculate the final results. Its formula is given in Equation (15).

$$MFM = \frac{\sum_{i=1}^{r}(FM)_i}{r} \tag{15}$$

— **Mean of the area under the ROC curve (MAUC)**
It is the average pairwise AUC value of all the pairs of classes. It can analyze the efficacy of imbalanced learning algorithms more accurately. Its formula is given in Equation (17).

$$MAUC = \frac{2}{r(r-1)}\sum_{i<j}(AUC(C_i, C_j)) \tag{16}$$

$$MAUC = \frac{2}{r(r-1)}\sum_{i<j}[A(C_i, C_j) + A(C_j, C_i)] \tag{17}$$

— **Kappa**
Although the accuracy metric is effective for binary dataset classification, the distribution of filled and empty classes in our training contextualized data tuples is uneven. Therefore, accuracy and the Kappa measure cooperate to prevent inaccurately predicted outcomes [80]. Its formula is given in Equation (18).

$$k = \frac{p_O - p_E}{1 - p_E} \tag{18}$$

— **Probabilistic AUC**
Because accuracy can lead to erroneous results, the more accurate metric AUC is used instead of accuracy. We need to update the concept of this measure for multi-class situations, because it was first proposed for binary-class imbalanced datasets. So, the Kappa measure and accuracy work together to prevent inaccurately predicted outcomes. For each pair of classes, a single value is computed, including one positive (minority) and the other as a negative (majority). Following that, the result's average is calculated. Its formula is given in Equation (19).

$$PAUC = \frac{1}{C(C-1)}\sum_{j=1}^{C}\sum_{k!=j}^{C} AUC(j, k) \tag{19}$$

3 Comparison with Existing Surveys

In this section, various approaches and results from previous research attempts in imbalanced data handling techniques are critically examined and summarized, with a focus on how to handle multi-class imbalanced data and concept drift issues in the context of fog computing. Through an examination of current surveys, this section seeks to pinpoint important findings, knowledge gaps, and prospects for future developments in the field. A few of them are discussed as follows:

Mercedes E. Paoletti [81] presented a thorough experimental analysis for imbalanced data in classification of hyperspectral data. The study had two goals: First, it reviewed oversampling techniques that were more appropriate for HS data and, second, it provided a more thorough experimental analysis and comparison. The comparison of oversampling methods in the paper was done based on several key criteria: (how the new synthetic samples are generated using the SMOTE algorithm, considering the proximity of minority class neighbors); selection of generator samples (how samples were chosen from the dataset to act as the basis for generating new synthetic data); use of classifiers (role of classifiers in identifying which samples or clusters should be used to generate synthetic samples); sample generation method (specific techniques used to create new samples from the selected generator samples); and location of new synthetic samples (where these new synthetic samples are positioned within the feature space after they are generated, which can impact the effectiveness of the oversampling technique). This work gives three experiments. First performs a comparison using several machine learning models (MLR, SVM, and shallow and **Deep multi-layer perceptron (MLP and DMLP)**). Different deep-learning models were compared in the second experiment. The third experiment evaluated the impact of the class imbalance problem on the models of semantic segmentation that are trained with different loss functions i.e, **focal loss (FL), cyclical focal loss (C-FL), asymmetric focal loss (A-FL)**, and **cross-entropy (CE)**. It highlighted the limitations of ADASYN and K-means SMOTE with restrictive constraints on the minimum number of samples per class. It also highlighted the need to generate a few more deep network mechanisms. First, it was noted that imbalanced datasets cause the classic cross-entropy loss function to perform poorly for minority classes. This has emphasized how crucial it is to address the class imbalance by utilizing balance-aware loss functions. Ultimately, the research has demonstrated that mIoU is a more appropriate metric for assessing performance on imbalanced datasets than overall accuracy. The author suggested expanding this work to include undersampling and oversampling in the future.

D. Devi [82] provided a review of undersampling techniques, then compared and contrasted a few methods of pure undersampling techniques, cluster-based undersampling techniques, and, finally, a comparative study of a few different hybrid undersampling techniques was provided. This study produced a list of a few points that future researchers can use to help them investigate the problem and come up with fresh ideas. The significance of a pattern was highly related to its **neural networks (NNs)** and their distribution properties. Combining an informative undersampling technique with an efficient clustering algorithm was very effective. Undersampling with ensemble learning and evolutionary algorithms can be used to achieve a tradeoff between accuracy and training time.

A survey on software fault prediction for imbalanced data was conducted by S. Pandey [83]. The training phase of a dataset determined the model's accuracy. Therefore, if there was a dataset fault, then it could result in issues with class overlapping, null values, or imbalanced classes. Because models built on faulty data could produce inaccurate predictions, software fault prediction focused on data quality. Thus, the most recent fault prediction algorithms in machine learning, deep learning, and ensemble learning were covered in this survey. SMOTE, a data sampling technique based on the literature, was widely used for software fault prediction.

Table 10. Comparison with Existing Surveys

Ref.	Imbalanced	Concept drift	Limitation
[81]	Yes	No	This survey primarily focuses on oversampling techniques, and other imbalance correction strategies such as cost-sensitive or algorithmic approaches are not discussed.
[82]	Yes	No	Computational cost of undersampling technique is not discussed. Certain undersampling techniques may become ineffective in practical applications for large datasets because of their high dimension costs.
[83]	Yes	No	This survey report focuses on specific methods (SMOTE) limiting exploring alternative or complementary approaches to addressing class imbalance issues.
[84]	Yes	No	This survey compares the techniques that only use f-measure and do not provide a comprehensive evaluation of other metrics.
[85]	Yes	No	There is not enough novelty or comparative analysis in this work.
[86]	No	Yes	It does not delve deeply into the specific methodologies, their strengths, weaknesses, or comparative performance.
[87]	No	Yes	It might make it more difficult to directly compare strategies, since it lacks a consistent evaluation metric for evaluating the efficacy of various concept drift handling techniques.

A. Sharma [84] presented a survey report. The methods for handling imbalanced data that were proposed by different researchers were listed in this survey in the following categories: data-level, algorithmic, hybrid, kernel-based, and cost-sensitive. Using a common dataset and set of classifiers, the approaches presented in this survey were compared using F-measure values. The analysis of these approaches led to the conclusion that SMOTE overcame the limitations of RUS and ROS.

A review of 17 research papers published between 2018 and 2021 was given by SJ Basha [85]. To address the issue of class imbalance, this survey offered the following imbalanced dataset classification methods: KNN, updated KNN with feature selection, Distributed deep learning, GAN, Gradient boosting, iterative expansion algorithm, KSAMOTE, IAdaBoost, RandomOversampling, RCT, Label enhancement technique, and oversampling with DL approach. These strategies addressed the problem of classifying imbalanced datasets.

This survey aims to address limitations by discussing various imbalanced data handling strategies beyond oversampling, including cost-sensitive and algorithmic approaches, while also addressing concept drift challenges. It plans to explore alternative methods to tackle class imbalance and concept drift effectively, evaluating techniques using diverse metrics for a comprehensive assessment. Furthermore, the survey seeks to improve understanding in the field by enhancing its analysis with more novelty, comparative studies, and a thorough investigation of methodologies in the context of multi-class imbalanced stream data in Fog computing.

4 Applications of Imbalanced Data Handling Techniques

The term "imbalanced class distribution" refers to the tendency of a dataset collected throughout the process to have more observation instances related to one class than to the other classes, and a dataset with such a property is known as "imbalanced data." The imbalanced data problem frequently arises during data processing in IoT applications. Under normal circumstances, it is challenging to collect enough samples of unusual conditions, and creating unusual conditions would be prohibitively expensive or dangerous. Imbalanced learning is a pressing subject that has been covered by numerous scholars, and here we discuss it in various sections.

Imbalanced data handling in various networks is explored in Section 4.1. Section 4.1.1 presents binary and multi-class imbalanced dataset handling in a wireless sensor network. For IoT networks,

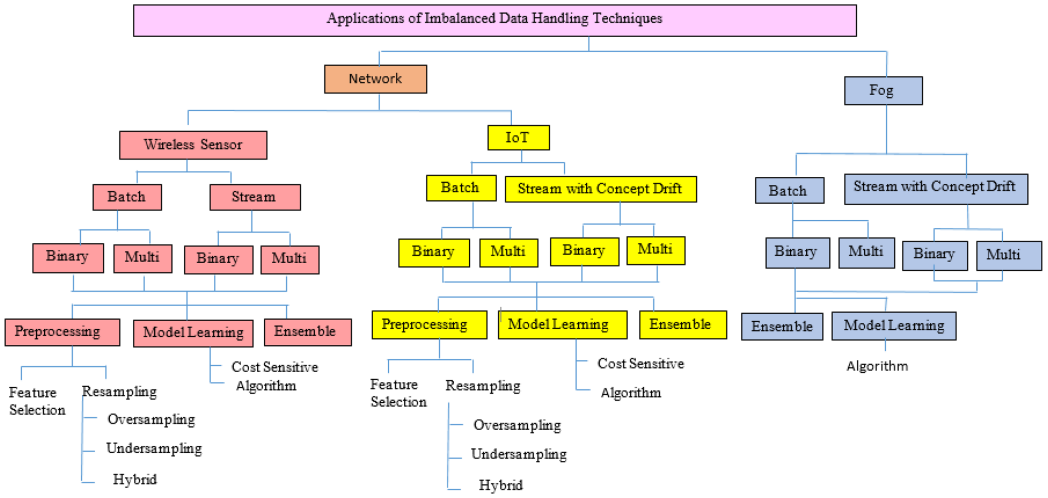

Fig. 4. Taxonomy of imbalanced data with concept drift.

the progress for the imbalanced IoT dataset handling is divided into binary-class and multi-class batch data. For streaming data, the imbalanced data stream handling for binary-classes and work for imbalanced multi-class data streams in the IoT network is further elaborated on in Section 4.1.2. The latest research on imbalanced batch and stream data handling in Fog computing is discussed in Section 4.2. To handle concept drift along with imbalanced data in the Fog computing environment, different techniques are also explored in this section. Figure 4 shows the taxonomy of the imbalanced data with concept drift.

4.1 Imbalanced Data in Networks

Many small- and large-scale enterprises that employ network services to carry out their everyday activities have recently benefited from technological advancements in terms of comfort and opportunities. It allows for exploring and exploiting several attacks by intruders or attackers. Today's escalating cyberattacks on networks lead to an imbalanced distribution of classes. These problems have been addressed using a variety of approaches. The following describes a few of these.

4.1.1 Imbalanced Data Handling in WSN. A **wireless sensor network (WSN)** is made up of a large number of low-power, battery-powered, and low-cost sensor nodes. As these sensor nodes are non-rechargeable and have minimal energy resources, they must be properly controlled to extend the network's lifespan [88]. When sensors create data, there is a potential that the data will be discontinuous, resulting in sparse data that is imbalanced. Imbalanced data processing for WSN is covered in several circumstances given below:

— **Activity Recognition**
Some actions occur more often than others in activity recognition datasets, resulting in an imbalanced dataset. The goal of Reference [89] was to solve class imbalance problems in automated activity identification from patterns of binary sensors in a smart home. Initially, publicly available datasets from three different households were used. The activities of an individual residing in an apartment were monitored using a wireless sensor network in which each sensor was connected to a node. A base station gathered the data, which was then labeled using a wireless Bluetooth headset and a software for voice recognition, as well as a handwritten journal or a PDA. The model recognized the activities based on the

binary sensor outputs. Instead of representing raw values of sensors, the author employed the "change point" representation (which assigns the value one (1) when the reading of sensor changes from one (1) to zero (0) or zero (0) to one (1)) and "Last" representation (which assigns one (1) to the "last" sensor that changes state until a new sensor changes state). All courses that lasted longer (idle and sleeping) were classified as majority classes, while others were classified as minority classes. In the trials, the SVM hyper-parameters (σ, C) were tuned. Several binary classifiers were trained using a multi-class CSVM. Finally, a learning approach combining multi-class SMOTE-CSVM and OS-CSVM was presented, with results showing that resampling methods were more efficient than CSVM, CRF, and CS-SVM in classifying multi-class sensory data.

— **Manufacturing Process**
Incomplete and missing values can be found in data obtained from semiconductor manufacturing processes in a real-world setting. This incomplete and imbalanced data gives biased results. So, Reference [90], used two steps to overcome this problem. Initially, KNN performed the missing value imputation. Then, using an Adaptive Synthetic Sampling technique and a 2-layer Feed-forward Neural Network as a classifier, they solved the imbalance problem by artificially introducing additional minority class samples and forecasting the faulty items. Although the suggested approach did not perform well with an incomplete dataset, it did obtain a high and tolerable identification performance with no bias.

— **Energy Consumption**
The sensors' energy consumption may become imbalanced and cause particular local nodes to deplete prematurely. In this scenario, typical classification methods are frequently found to be erroneous and optimized. [91], suggested a novel technique that extended the stream classification algorithm to the analysis of WSN to lessen the negative impact of the imbalanced class of data. This technique was low on resources and did not necessitate any preprocessing, which would have required going through the entire database. It employed naive Bayes predictors at the leaf nodes of the decision tree to limit the influence of imbalanced classes. A stream classifier was used instead of a standard classifier in this study.

— **Cluster-based Routing**
In a cluster-based wireless sensor network, non-uniform node distribution produces uneven energy consumption across nodes. It is a critical issue impacting network services. As a result, [88], presented a cluster-based routing protocol for WSNs with non-uniform node distribution to address this issue. This protocol included the energy-aware clustering algorithm EADC, which built clusters of even sizes, and the cluster-based routing algorithm, which adjusted the intra-cluster and inter-cluster energy consumption of cluster heads to balance the energy consumption among cluster heads. By compelling cluster heads to accept nodes with great energy and fewer nodes as next hops, load balancing among cluster heads is achieved, resulting in an even distribution of energy consumption across nodes and a rise in the number of nodes. According to a review report presented by [92], a few pieces of work have concentrated on imbalanced data handling in WSN. This piece of work provided suggestions for extending traditional imbalanced data handling approaches to a WSN, especially K-fold cross-validation, ensemble resampling datasets, assigning weights to attributes, cost-sensitive learning, and combined class methods.

— **Intrusion Detection System**
An intrusion detection system monitors network traffic in real time to distinguish between malicious attacks and normal traffic. Because it must properly detect all threats, even in the presence of a tiny percentage of intrusion data, [93] focused on an imbalance problem in the intrusion dataset. The study used four prominent classification approaches to examine the

Table 11. Comparison of Imbalanced Data Handling Techniques in WSN for Batch and Stream Data

Technique	Strategy	Class	Concept drift	Dataset	Tools	Parameters	Ref
Soft Margin SVM	Sampling	Multi (Batch)	No	Datasets of 3 Houses	Matlab	Accuracy	[89]
KNN-ADASYN-FNN	Algorithm	Binary (Batch)	No	Secom, Secom1, Secom2	-	Recall, Precision, F1-measure	[90]
OVFDT+FL	Algorithm	Multi (streams)	No	LED24, Connect-4, Waveform 21, RBF, RT, COVTYPE	MOA simulator	Accuracy, ROC, compact DT size	[91]
Cluster based inter-cluster algorithm	Algorithm (Clustering)	Binary (Batch)	No	NS-2 simulator	No	Sensor field, BS location, the initial energy of nodes, # of nodes, data packet size	[88]
Dataset evaluation	Survey Report	Multi (Batch)	No	NSL-KDD	Weka	Accuracy	[93]
Correlation-based scheme	Feature selection and Clustering	Multi and Binary (Batch)	No	SatimageBreast Wisconsin, Glass, Yeast, Phoneme	Weka, NS-2 simulator	Accuracy	[94]
WSVM	Algorithm	Binary and Multi (Batch)	No	Dataset of 3 Houses	MATLAB, LibSVM	Accuracy, class accuracy	[95]

Table 12. Concept Drift in WSN

Technique	Key Points	Dataset	Tools	Parameters	Ref
FedConD	It addresses concept drift on local devices and uses a communication strategy on the server side to select local updates.	Air Quality, extrasen-sory	FedConD framework	Regularization parameter of the objective function on each local device	[96]
Angle Optimized Global Embedding (AOGE)	Project variance and projection angles are used to analyze the principal components, and the change in subspace is used to detect the occurrence of concept drift.	Synthetic dataset (Circle, Sine, and Line)	AOGE and PCA	Projection variance and projection angle, constraint parameter for determining the occurrence of concept drift	[97]
Hybrid Batch Online Stacking Ensembles integrated with GA	GAHS uses an online machine learning calibration function or functions that are updated on a regular basis for the entire network in addition to batch machine learning algorithms.	Air Quality Low-Cost Sensor Network (AQLCSN)	-	Pearson correlation coefficient, determination coefficient (R^2), root mean squared error (RMSE), mean absolute error (MAE), relative expanded uncertainty (REU)	[98]

NSL-KDD dataset and discovered that severely imbalanced classes were not successfully categorized. Random Forest, which is an ensemble-based classifier, performed well for a few minorities and the remaining majority classes but struggled with severely imbalanced classes.

Table 11 shows a comparison of imbalanced data handling techniques in WSN

Numerous studies on concept drift in WSN enhance the prediction accuracy and adaptability of WSN systems in dynamic environments. Below are some of them.

4.1.2 Imbalanced Data Handling in IoT Network. The IoT is the network of computing-capable and Internet-connected devices that are typically not thought of as computers. Because of the maximum use of these smart devices across numerous networks (home, business, military, etc.), a number of problems have emerged, and one of them is data imbalance. There has been extensive research on the imbalanced data in the IoT, which has been divided into binary and multi-class data.

Table 13. Imbalanced Batch Data in IoT Network for Binary Classes

Technique	Strategy	Datasets	Tools	Parameters	Ref
Anomaly-based IDS	Resampling	CIDDS-001	Weka, MATLAB, Keras	Accuracy	[103]
DeL-IoT	Ensemble, Feature Extraction	Testbed data as well as Benchmark data	-	F1-measure, MCC	[3]
CSSAE	Cost-sensitive	KDD CUP 99 and NSL-KDD	DARPA's evaluation program	Accuracy, Recall, Precision, F-measure, False Alarm Rate	[100]
Imbal-OL	Resampling	CIFAR-10 AND CIFAR-100	IoT board (Google Coral Dev board, Intel Movidius NCS)	Accelerated Raspberry Pi 4, and NVIDIA Jetson Nano)	[101]
Framework for handling IoT datasets	Ensemble	-	Keras, PySpark	Accuracy	[102]

Deep learning models are trained progressively over time. They reduce their static training with all of the data. To deal with imbalanced data, combining class-incremental learning with the IoT is a newly introduced notion that is still in the early stages of development. The main feature of Reference [99]'s data sampling algorithm was the capability of sampling data from novel classes without using hyperparameters by automatically choosing the number of samples required per incremental training session. A few studies for handling imbalanced data in the IoT are mentioned below:

(1) **Binary-class Batch Data Handling:**
In the case of IoT data, the security risks have sharply increased recently, and the attack methods used by the attackers are frequently changing and improving. Additionally, the frequency and complexity of imbalanced class distributions in most datasets point to the necessity for additional research. As far as binary-class imbalanced data in IoT networks is concerned, Table 13 gives various research approaches. The DeL-IoT technique, proposed by Reference [3], was introduced to detect SDN-based IoT anomalies. It also addressed the issue of multi-class as well as binary-class data being imbalanced. In another technique of **cost-sensitive stacked auto-encoder (CSSAE)** [100], stacked autoencoder with the Sigmoid function employed in the initial stage. The SAE of the second phase, however, used Tanh as an activation function. The two SAEs' learned features were merged. This technique was used for both binary and multi-classes. The technique of Reference [101] was suggested as an **OL (Online Machine Learning)** plugin that would process actual IoT streams and after that send them to the learner. After the whole process, the local on-device model is updated. It worked on data streams. A framework for handling IoT datasets [102] adopted Adam optimization, an extension of **stochastic gradient descent (SGD)**, which has lately gained wider recognition for deep learning and IoT applications. It also worked for batch and stream data. According to Reference [103], it was observed that dataset characteristics matter, but class distribution had little effect on the classification issue.

(2) **Multi-class Batch Data Handling:**
A multi-class imbalanced dataset is considered an imbalanced dataset when there are more occurrences of a few of the classes in the training set than there are of other classes. It affects how effective machine learning algorithms work. In comparison to the algorithms of ML, deep learning techniques perform well when learning from huge volumes of data, but their performance suffers dramatically when learning from imbalanced data. For measuring the performance of multi-class datasets, macro metrics are used to independently calculate the metrics for each class, and after that, it takes the average for multi-class imbalanced data. Various models have been suggested for multi-class batch data. For instance, the model

Table 14. Imbalanced Batch Data in IoT Network for Multi-classes

Technique	Strategy	Datasets	Tools/frameworks	Parameters	Ref
RITIDS	Algorithmic	CICIDS2017, BoT-IoT	Weka, MySQL RDBMS	Accuracy, Detection rate, False Alarm Rate, Time Overhead	[104]
GAN Model	Generative Adversarial Nets	CICIDS 2017	Python, TensorFlow, Scikit-learn	Recall or F1-Score	[105]
CSSAE	Cost-sensitive	KDD CUP 99 and NSL-KDD	DARPA's evaluation program	Accuracy, Recall, Precision, F-measure, and False Alarm	[100]
DAMSID	Ensemble	SEA	-	Accuracy	[106]
Comparative analysis	Resampling	KDD99, UNSW.NB15, UNSW-NB17, UNSW-NB18	Spark MLib	Macro precision, Macro recall, Macro F1-score	[107]
Adaptive boosting-based model	Ensemble	NSL-KDD, Synthetic, KDD CUP99, DS2OS	Python Anaconda, Spyder IDE, Pandas, Imblearn, Numpy, Matplotlib	Sensitivity, F1-score, ROC-AUC	[108]
GWO–PSO–RF NIDS	Feature Extraction and Resampling	KDD CUP99, NSL-KDD, CICIDS-2017	Python, Anaconda Navigator	Accuracy	[109]

proposed by Reference [104], was made up of three classifiers, two of which run simultaneously and feed the third. Evaluation results revealed that this hierarchical model beats various popular and contemporary machine learning algorithms. Reference [105] displays that "when Random Forest was used to classify data after GAN resampled it, its performance outperformed that of a single RF alone." The model of Reference [100] recommended that the issue of class imbalance in IDS could be solved by a cost-sensitive stacking auto-encoder. It was used both for binary as well as multi-classes. The issues of classification with concept drifts and imbalanced data were simultaneously addressed in Reference [106]. To determine the most effective methods for handling imbalanced data, six separate datasets were subjected to five different resampling techniques in Reference [107]. Reference [108] presented an ensemble learning-based approach with the SMOTE. It successfully handled both the imbalanced nature of the data and the anomaly prediction in the IoT network data DS2OS. To achieve maximum attack detection accuracy, the method was suggested by Reference [109], using **Particle Swarm Optimization (PSO)** and **Grey Wolf Optimization (GWO)** for extracting meaningful IoT network features that were then given to a **random forest (RF)** classifier. It worked for both the binary and multi-class.

Table 14 summarizes these recent approaches for imbalanced data in IoT for multi-classes.

(3) **Binary-class Stream Data Handling with Concept Drift:**

The continuous arrival of data that must be analyzed at once on each scan causes challenges for stream data mining. Moreover, a number of challenges have to be taken into account while dealing with streams of imbalanced data. The concept drift is one of these challenges. The researchers offered different techniques to handle these challenges. The technique mentioned in Reference [110] processed the fixed-size chunks twice: once using oversampling and once using an ensemble of prediction models. It performed better with a shorter time delay and can be employed with dynamically imbalanced data streams. According to the technique proposed by Reference [111], to a large extent, CtRUSBoost surpassed all of its competitors in detecting transactions as normal or fraudulent. The technique presented in Reference [112] did not take as much time as other evaluated algorithms. It employed a resampling method that concurrently took concept drift into account and followed that with

Table 15. Imbalanced Data Streams along with Concept Drift in IoT Network for Binary Classes

Technique	Strategy	Concept Drift	Datasets	language/Tools/ frameworks	Parameters	Ref
ICMS	Resampling	Yes	Synthetic data (static and dynamic Imbalanced Ratio)	MOA, Stream-learn, Scikit-multiflow	Accuracy score, G-mean score	[110]
CtRUSBoost	Ensemble	No	3 datasets of credit card fraud from Kaggle	-	Sensitivity, specificity, precision, F1-score	[111]
GRE	Ensemble	Yes	SEA, Radial basis function, Hyperplane, Electricity pricing (Elec)	JAVA, MOA	Accuracy, Recall, F-measure, G-mean, AUC	[112]
PWIDB	Automated data re-balancing	No	(ECC) dataset, UCI Adult dataset	-	AUC-ROC, F1-score	[113]
PWPAE	Ensemble	Yes	IoTID20, CICIDS2017	Python, Scikit-Multiflow	Accuracy, Precision, Recall, F1-score	[114]
Two-layer ensemble Kohonen nets	Feature selection	No	Adult census dataset, Bank marketing dataset	Apache Spark	Accuracy	[115]

an ensemble update mechanism and a detailed analysis of both the real-world and synthetic datasets. The framework proposed by Reference [113] used a batch-incremental process to handle the demands of dealing with imbalanced data streams dynamically. Another framework proposed by Reference [114] was a drift-adaptive framework for finding anomalies in the IoT. It was built by using an ensemble of cutting-edge drift adaptation techniques. The technique given in Reference [115] was more concerned with identifying and separating areas where the minority class was concentrated. According to a survey report by Reference [116], the majority of solutions were proposed for datasets with binary-classes and not with multi-classes. However, before inclusion, multi-class datasets need to be transformed into binary-classes. Table 15 summarizes these techniques for imbalanced data streams having concept drift in IoT for binary-classes.

(4) **Multi-class Stream Data Handling with Concept Drift:**
Attacks make data streams imbalanced and make it possible for the concept of a data stream to change over time. To deal with this problem, a few researchers have presented their work. For example, a survey report given by Reference [117] assessed both imbalanced data strategies' effectiveness and demonstrated how machine learning algorithms manage streams of network traffic. A method in Reference [118] was suggested to change the low-weighted data in the contextual information while keeping the weighted data in the acquisition of contextual information, as opposed to applying uniform oversampling or undersampling. Reference [119] expanded the concept drift procedure into imbalanced class circumstances by creating an adaptable learning algorithm with a Windows-based methodology. Reference [120] took two steps. First, cost-sensitive learning was employed in the process of feature selection. Then, a cost-sensitive weighting schema was designed to update the weight of the latest data block. Table 16 summarizes different approaches for dealing with imbalanced data streams and concept drift in the IoT for multi-classes.

4.2 Imbalanced Data Handling in Fog Computing

The cloud services are pushed to the network's edge through a distributed computing model called *Fog*. Fog computing techniques have been proposed to reduce latency and computing load. The pieces of work presented by different researchers for batch and stream data handling show the importance of this field. A few of these research works are given below:

Table 16. Imbalanced Data Stream along with Concept Drift in IoT Network for Multi-classes

Technique	Strategy	Concept Drift	Datasets	language/Tools/ frameworks	Parameters	Ref
Survey on network traffic stream analysis	Tree-based algorithm, Ensemble	Yes	UNSW-NB15, NSL-KDD, UNSW 2018	Scikit-multi-flow, Python package	Accuracy, Kappa, Cohen's Kappa	[117]
Smart switchboard imbalanced data	Data-level	No	Data reported by sensors	Smart switchboard	-	[118]
FP-EStream	Algorithmic	Yes	3 entries from parking lot of University of Essex	MOA, NetLogo	Speed	[119]

(1) **Batch Data Handling in Fog Computing:**
At present, a few works have been done in imbalanced data handling in Fog computing. One of the key contributions of this research is to present the work done for imbalanced batch data handling in Fog computing. For example, a Fog-based unsupervised machine learning prototype for a large volume of data analysis was developed in Reference [121], which replaced the initial deployment of machine learning modules and signal processors in the cloud for processing physiological data. The Parkinson's disease patients wore smartwatches that collected speech data to assess their speech impairments. The speech data was sent from a smartphone or tablet to a Fog computer. K-means clustering was utilized to process some features on the Fog computer. Reference [122] presented a comparison of cloud and Fog computing, some challenges, and open issues in Fog computing. This research also analyzed Fog computing deployment in intelligent logistic centers and proved that deployment of Fog computing improves energy efficiency, reduces latency/costs, and supports mobility.
Another study discusses how machine learning can be used to perform more accurate fault detection when collecting data. Real-time fault detection has a few issues. One of the issues is an imbalanced class, which causes extreme difficulty in using machine learning models in real-world settings. In the case of an imbalanced class, where the number of instances of one class is greater than that of another, the machine learning model is overfitted towards numerous examples and causes performance degradation. Because the fault does not occur frequently, most data occur in a normal state, making it a serious situation. To overcome the class imbalance, the methodologies that are adopted by Reference [123] are the computing architecture solution method and the algorithm solution method. Table 17 summarizes data handling in Fog computing.

(2) **Stream Data Handling in Fog Computing:**
Data stream processing and analytics are used in many Fog applications. These are widely used in the cloud but have yet to be thoroughly examined in the context of Fog architecture. By examining the common aspects of numerous typical applications, Reference [132] described the main principles and architecture of Fog data streaming.
Data streams in the IoT environment are made accessible in unlimited flows, continuously produced at high speed, and their behavior changes over time rather than remain stationary. These qualities of the data make it known as "real-time big data" and give it several Vs (volume, velocity, variety, and veracity). These characteristics are related to the huge volume of continuously generated data, the high speed at which several devices generate IoT data, the variety of devices and data sources, and the effect of data by environment and noise

Table 17. Imbalanced Batch Data in Fog

Objective	Imbalanced	Fog Nodes	Privacy/Security	QoS/QoE	Ref
It explored machine learning on Fog devices with limited resources	No	Edison and Raspberry Pi	-	Resource Management	[121]
Ranking-based job scheduling system from the most suitable Fog nodes to the least suitable ones	No	Any Fog provider	User satisfaction, Energy, Latency	MatLab	[124]
A dynamic multi-goal approach manages the energy of IoT-based wearable systems	No	Local Central device, Gateway device	-	Energy Efficiency	[125]
Anomaly detection using data-driven network intelligence	No	-	Accuracy, Low latency	-	[126]
The use of Fog computing in the logistics system	No	Gateways and Fog devices	-	Mobility, Energy Efficiency, Reduce latency, cost	[122]
In a Fog computing (FC) scenario, this intelligent analytical model was used to allocate and select healthcare IoT data packets	No	Gateway	-	Latency, Network usage, Ram consumption (MB), (Net-Beans and Spyder)	[127]
It provided a 3-tier Architecture for reducing network latency in Healthcare IoT	No	Gateway	-	Latency	[128]
Developed three IoT network architectural designs for the LoRaWAN cloud architecture, then optimal is selected.	No	Gateways	-	Low power consumption and Location awareness	[129]
IoT, Fog, and Cloud integration (iIFC) enables optimized application performance	No	Gateway	Security	Energy, Transport health, etc.	[130]
An algorithm solution method and a computing architecture solution method were used to overcome the class imbalance problems.	Yes	Fog cluster for each group of sensors	Reduces performance degradation and computational load	F-measure, G-mean	[123]
To develop a lightweight anomaly detection model for deployment on Fog nodes	Yes	Fog devices	Security	Precision, Recall and F1-score	[131]

transmission. The important characteristics of mining data streams are the use of short-term memory as a queue to store subsets of data and the use of a limited amount of memory. Other characteristics include maintaining linear spatial and temporal complexity to operate within the execution time and providing a solution when required.

Reference [128] offers the Fog computing 3-tier architecture that consists of fuzzy logic and reinforcement learning. This architecture minimized latency by utilizing machine learning and virtualization approaches. The first layer was the IoT layer, which contained sensor devices that produced data. The classification of the data was done using a fuzzy inference technique. The classified data was sent as streams in a Fog computing environment to a real-time analyzer like Apache SPARK. The second was the Fog layer, which used distributed reinforcement learning to select that data from the classified data that is most time-sensitive. After that, it performed the virtualization of the Fog server for data allocation. An iFogsim and a Spyder editor tool based on Python were used for simulating the Fog computing-based architecture and analytical models and for analyzing the performance of the architecture and the algorithm. In the end, the third layer was the cloud that contained data for future use.

Reference [133] presents a new **hybrid security strategy (HS2)** that is merging the strengths of steganography with cryptography to create a method for protecting the Fog. The first contribution of HS2 is a new encryption technique that depends on n-blocks of **linear feedback shift registers (LFSRs)** merged with a subtractor or adder for creating the strong key for each and every block. Then, all the blocks were combined to generate a final key. The second contribution is the steganography methodology based on upgraded **discrete wavelet packet transform (DWPT)** that is used for embedding the encrypted secret image into a cover video. The findings show that this strategy outperformed the recent security strategy.

To implement the proposed approach in Reference [134], platforms may be used as sensor nodes, and one of them is the Raspberry Pi. Compared to traditional sensor nodes, these sensor nodes provide better computational power. These sensor nodes create the encrypted XML documents by applying the suggested algorithm of encryption. The algorithm applied for encrypting the contents and features of the specified XML elements must be executed by the sensor nodes to produce the encrypted XML documents. To do this, they need to use a secret channel to obtain common parameters, encryption functions, and secret keys from the server. In this study, Fog computing is used. Fog nodes can execute lightweight computational services like aggregations. XML filtering processes the XML streams, but it concentrates on one stream at a time and infrequently handles several streams at once. Moreover, it proposed a model that expanded the XML encryption standard to include data stored in sensors as strings and numeric types. It efficiently filtered the matched streaming data and performed summation at Fog nodes. It also performed filtration operations without decryption at Fog nodes. This model rapidly processed numerous encrypted XML streams produced in parallel by sensors without disclosing private information to the subscriber. In another technique, XML streams were generated by sensors. To evaluate the proposed approach, the PC environment or Rasberry Pi platform was used to implement the Fog node. As compared to a PC, the Raspberry Pi carries less computing power, but, still, its execution time remains satisfactory. However, the maximum use of the concurrent XML filters, because of the limited computation resources of the Raspberry Pi, always results in resource competition. The efficiency was obtained by increasing the number of concurrent XML filters.

Fog computing supports large amounts of stream data generated in IoT scenarios. A new FOT platform is introduced by Reference [135] for handling stream data in Fog computing. It is used in Fog to process and analyze real-time stream data from the IoT. Its main benefit was to reduce internet usage. Through the detection of changes in data behavior and the reduction of a huge amount of data transmission over a network infrastructure, online data modeling can be made possible. The occurrence of such unpredictable and unexpected changes motivates the design of the concept drift detection method. A method named **Cumulative Sum (CUSUM)** is adopted in this study due to its low complexity computations and is considered to be memoryless.

Reference [136] proposes a five-tier architecture in which the stream data initiating from various IoT devices is moved to IoT gateways using various protocols (MQTT, CoAP, Zigbee, WiMax, etc.) of communication. These gateways perform aggregation of data, and then, for further processing, they publish it to Fog nodes. A modern and frequently adopted distributed messaging system called Apache Kafka and a stream processing engine called Apache Storm are the components of the Fog nodes.

For successful delivery of multimedia broadcasts, reliable content delivery, scalability, and video-stream quality must be ensured. The improvements in routing protocols and topologies improve reliability, scalability, and the quality of sharing information experiences.

Reference [137] proposes a collaborative routing protocol for video streaming ad hoc network that is dependent upon cluster architecture, and it uses Fog storage services to minimize content sharing. This model performs the calculation of a collaborative gateway to rank each vehicle with respect to the **Gateway Quality Indicator (GQI)**. Based upon the values of GQI, a routing table is defined that is built for each vehicle in the cluster of V2V communication. The vehicle collaboration is executed in the cluster for reducing the irrelevant data exchange. It is not necessary for all vehicles in a cluster to share the same live video, because irrelevant information will affect network performance. So, the algorithm, through cluster formation, finds the vehicle that has the best GQI, and the same vehicle becomes the collaborative gateway that streams the video via **Vehicle-to-Vehicle-Infrastructure (V2I)** communication. Table 18 summarizes various approaches for stream data handling in Fog computing.

(3) **Concept Drift Handling in Fog Computing**

Machine learning models that are trained on historical data and become out of date for data from the real world are referred to as experiencing "drift," which is a shift in the statistical characteristics of the data. This means that the machine learning models that have been trained gradually deteriorate and lose their ability to utilize patterns to make predictions in the future. The term drift can be used as data drift, which is the change in data distribution; or it can be used as concept drift, which is the change in the objective or goal. Concept drift involves changes over time, requiring models to adapt to maintain accuracy continuously. The concept drift can be detected using concept drift detectors, sliding windows, online learners, and ensemble learners. The approaches listed below in Table 19 for managing concept drift are proactive, providing valuable insights through advanced machine learning techniques and optimization strategies. These methods demonstrate effective handling of concept drift, ensuring models remain accurate and adaptive in dynamic environments. Reference [147] uses LSTM models for detecting sudden and gradual concept drift in the cloud domain using a genetic hyper-tuning drift detector, leading to improved performance and more efficient resource allocation. Reference [148] handles concept drift within non-stationary spatiotemporal data streams. BOASWIN, adaptive XGBoost-based model with the BO-TPE hyperparameter optimization strategy, has become a powerful tool for spatiotemporal data analytics. This model improves classification accuracy and remains responsive to continuous and predictable changes in data distribution by dynamically adjusting window size based on detected drift. Reference [149] proposes a framework for dynamic streaming data analytics. In this work, pattern changes in the data streams during incremental learning are adapted using an **optimized adaptive and sliding window (OASW)** that efficiently manages memory and time constraints.

Concept drift in Fog computing, caused by dynamic network conditions and system updates, alters data distributions over time. This requires models to adapt for accurate predictions. To present a detailed evaluation of different machine learning and AI models used in Fog computing to mitigate concept drift, Table 20 highlights the design, implementation, and critical analysis of each model, emphasizing proactive approaches. This comprehensive comparison ensures a clear understanding of the strengths and limitations of each method, thereby facilitating more informed decisions in the application of Fog computing technologies. In the given below different concept drift handling techniques in Fog computing are mentioned. In Reference [150], Fog-computing-based concept drift detection is combined with cloud-computing-based process mining. The proposed work actively detects and responds to concept drift, preprocesses the data locally, and maintains multi-version process models, which results in efficient and timely process mining for mobile applications.

Table 18. Stream Data Handling in Fog Computing

Technique	Imbalanced	Key Points	Tools	Parameters	Ref
Aggregation and filtering model on XML streams	No	It provides a solution for Fog computing applications where maintaining the privacy of sensor data is a major concern	Raspberry Pi Platform	Privacy	[134]
3-Tier architecture for latency reduction	No	It minimizes latency	iFogsim, Spyder Editor	Latency	[128]
HS2 for reliable video Streaming	No	Low latency, Low network utilization, and no need for constant internet connection	Apache Flink, Spark, H2O	Low latency, low network utilization	[135]
Low-power portable metagenomics device analysis	No	The suggested method enabled instantaneous data analysis and sequence mapping as soon as the results were available	Python 2.7 and Karaken	Time	[138]
Architecture for traffic modeling and prediction services	No	It shows better behavior than its predecessors, even when connectivity concerns arose	Docker container	Time	[139]
Anomaly detection framework	No	Latency-sensitive applications might considerably benefit from a lightweight framework capable of continually and online identifying irregularities in the performance of various activities	Microsoft Azure	Time in user and kernel mode, bytes read and write to disk, iowait, bytes read, and write like system call	[140]
T3-Scheduler	No	The average throughput increased by 25% and 12%, respectively, as compared to the default and resource-aware scheduling strategies	Apache Storm	Throughput, resource utilization	[141]
Nornir a C++-based framework	No	It is flexible to implement different algorithms without explicitly interacting with applications	PARSEC	Throughput, latency, completion time, power consumption, energy	[142]
Viper	No	A communication module connected with the stream processing engine's communication layer improves parallel thread coordination during data analysis	Apache Storm	Throughput, latency, and energy efficiency	[143]
Hierarchical distributed architecture for elastic DSP application	No	Unlike threshold-based technique, the RL-based solution may account for various QoS metrics allowing the user to weigh the relative relevance of each measure	Apache Storm	Response Time	[144]
PiCo: new C++ API with a fluent interface	No	Compared to Spark and Flink, this new framework can achieve superior execution time while using less memory, making it ideal for resource-limited devices	Apache Spark, Apache Flink	Throughput, execution time	[145]
Edge-Fog-Cloud Architecture	No	If each edge-Fog-cloud resource is considered separately, then it will be unable to manage the data life cycles of IoT applications without sacrificing functionality or performance	RabitMQ, Cisco Kinetic, Scikit-Multiflow, Python,	-	[80]
Tracing framework	No	Presented solutions were capable of tracing with less coding and execution time	Apache Spark	Throughput, processing time	[146]

Table 19. Concept Drift

AI Model	Design and Implementation	Critical Analysis	Ref
LSTMDD is focused on cloud rather than Fog computing. It presents a proactive approach to handling concept drift in dynamic environments using advanced ML techniques and optimized LSTM models.	This model incorporates mechanisms to handle non-Gaussian distributed cloud data efficiently. It is optimized to improve performance in detecting anomalies that manifest as gradual and sudden drifts.	Its computational intensity and the requirement for sufficient historical data to train effectively might pose challenges in rapidly changing environments.	[147]
BOASWIN-XGBoost (Bayesian Optimized Adaptive Sliding Window and XGBoost) proactively prepares to detect and handle drifts but reacts by adjusting and retraining when actual changes are detected.	It uses an optimized version of XGBoost for classification, where the parameters of XGBoost are fine-tuned using Bayesian Optimization with a Tree-structured Parzen Estimator (BO-TPE).	This model effectively handles sudden, gradual, and recurring drifts. It shows improved performance in classifying streaming data by adapting the window size dynamically based on the drift detected.	[148]
Optimized Deep learning model and Adapting sliding window technique	It allows lower latency in data processing and quicker adaptation to changes in data streams. The algorithm detects changes by controlling the size and shift of the window.	Benefit of rapid response and localized data processing, limited computational resources. Managing window size and shift parameters may require fine-tuning	[149]

Table 20. Concept Drift in Fog

AI Model	Design and Implementation	Critical Analysis	Ref
A concept drift adapting algorithm is used to Integrate Fog computing for accurate log preprocessing with lower overhead and cloud computing for processing mined log. This approach actively adapts to changes, specifically, concept drifts due to the evolution of mobile applications.	Concept drift detection methods are used in the cloud computing layer to handle transitions from one version of a mobile application to another.	This integration allows for efficient preprocessing close to the data source (fog) and robust, scalable processing in the cloud. The concept drift adaptive algorithm enables real-time updates to process models, capturing the evolving nature of mobile app usage and operations.	[150]
This approach uses Wavelet Transform for data decomposition, allowing the capture of essential features while reducing data redundancy. Concept drift detection methods adapt to changes and optimize the use of network and computational resources.	Implemented in the FoT-Stream platform, which processes and analyzes data streams from IoT devices in real-time within the fog computing layer.	It reduces the amount of data transferred over the network by focusing only on significant changes, which optimizes both computational resources and network bandwidth.	[135]
Fog-DeepStream offers an incremental approach to efficiently model data streams in Fog Computing environments. This approach detects and adapts to changes in the data stream, allowing for timely model updates and predictions of evolving patterns.	It uses Wavelet Transform for data reduction, Concept Drift detection for model updates, and integrating Deep Neural Networks for enhanced system behavior understanding.	The effectiveness and scalability of this approach may require further validation across diverse IoT applications to assess its practical utility and performance in complex scenarios.	[151]

Table 21. Imbalanced Data Stream along with Concept Drift in Fog Computing

Technique	Imbalanced	Key points	Datasets	Parameters	Ref
Attentive Federated Learning	No	In single and multiple Fog drift scenarios, the model reduced mean absolute errors by roughly 20% compared to the baseline federated averaging approach.	FG trace data collected from Irish mobile operator in 2020	Mean Absolute Error (MAE)	[152]
Fog-computing-based concept drift adaptive process	No	It solves log incompleteness problems and provides process model evolution analysis in the mobile applications.	Two generated datasets	Precision, Recall, F1-measure	[150]
Concept drift adaptation technique in distributed environment	No	The review and implementation of the most recent concept drift (CD) detection methods were performed for time-series analysis in a distributed environment.	ELEC2, Fingrid	Mean absolute percentage error (MAPE)	[153]
DSPLE	No	To manage a dynamic IoT system, it deals with a change in stream data behavior.	-	Accuracy, Kappa	[154]
Tab Transformer	Yes	This approach uses a custom Tab Transformer for addressing multi-class imbalance and achieves high accuracy.	UNSW-NB15	Precision, Recall, F1 Score, Support	[131]

Reference [135] reduces the amount of data transmitted on the network, which allows online data modeling by detecting changes in behavior and reduction of internet usage. The framework proposed in Reference [151] continuously monitors data, efficiently manages it, performs incremental learning, and processes data faster by handling data near its origin (such as at the edge of the network), which facilitates timely analytics and decision-making. The concept drift in Fog computing has received little attention, yet it is still necessary to address multi-class imbalanced data with concept drift in the future. A few examples of concept drift in Fog computing are given in Table 21.

While the research on concept drift in fog computing does not explicitly focus on imbalanced data, it implicitly handles imbalanced data through methods for adapting to concept drift. These approaches often account for the changing nature of data streams, which can include shifts in class distributions, thereby addressing imbalanced data indirectly.

Figure 5 gives an architecture diagram that shows the overall data flow and processing stages involved in handling imbalanced data across IoT, Fog, and cloud. The processes include data collection, preprocessing, detection of imbalanced data, concept drift detection and adaptation, data transmission, and subsequent analysis and visualization.

5 Analytical Discussion

The researcher suggests using different imbalance correction approaches based on specific scenarios in Fog computing environments. It is imperative to customize the choice of solution to the characteristics of the dataset, the available computational resources, and the objectives of the application. For instance, undersampling techniques may be more suitable to address the class imbalance effectively in situations where the dataset is heavily imbalanced with limited computational resources. However, in scenarios where preserving information from the minority class is crucial, oversampling methods like SMOTE could be more beneficial. Cost-sensitive techniques prove valuable when misclassification costs vary between classes, allowing for a more customized approach to handling imbalanced data. Ensemble methods, such as combining multiple classifiers,

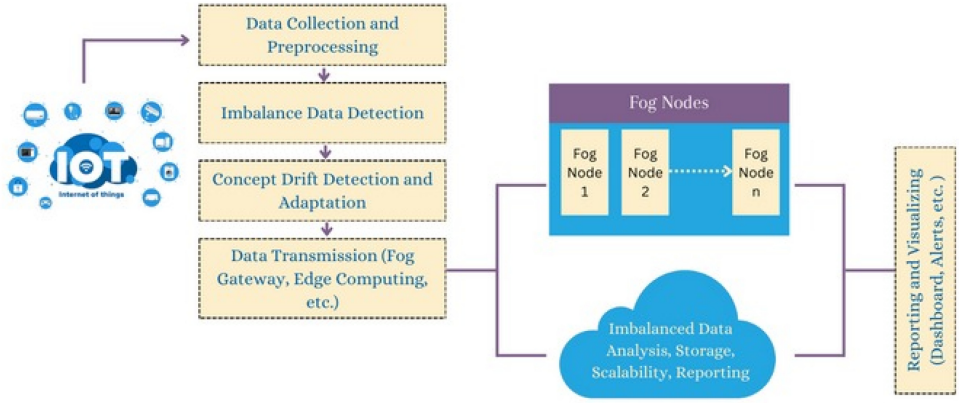

Fig. 5. Architecture diagram.

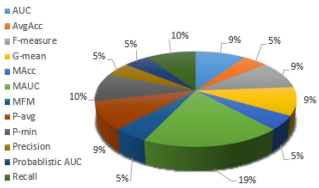

Fig. 6. Metrics for multi-class general form of data.

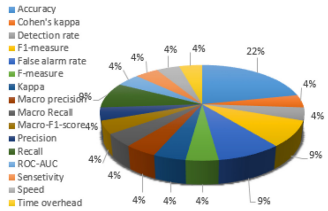

Fig. 7. Metrics used for multi-class data in IoT.

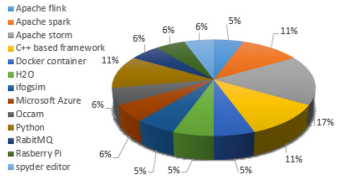

Fig. 8. Stream processing tools in fog.

can be effective in scenarios where a combination of techniques is needed to improve classification performance. By carefully assessing the specific needs of each scenario and understanding the capabilities of various imbalance correction approaches, practitioners can make well-informed decisions when selecting the most suitable solution to enhance data processing in Fog computing environments. The data is primarily categorized into batch and stream data. The researcher has analyzed the current binary and multi-class imbalanced data handling approaches for these two types of data. The imbalanced data processing categories along with concept drift, their contributions, the tools used, and the metrics employed in these studies provide the basis for the analytical investigation. Figure 5 displays the evaluation metrics that are adopted in a few recent pieces of research on multi-class imbalanced batch data. The most popular metric MAUC is used in 23% of the study; 12% P-min, 12% P-avg, 11% AUC, 6% MFM, 6% MAcc, 6% G-mean, 6% F-measure, 6% Recall, 6% AvgAcc, and 6% probabilistic AUC.

Figure 6 shows that the accuracy is 22% in the metrics that are used in multi-class IoT. Other metrics include recall 9%, false alarm rate 9%, f1-measure 9%, Precision 4%, f-measure 4%, G-mean 4%, Kappa 4%, Macro precision 4%, macro recall 4%, macro-f1-score 4%, ROC-AUC 4%, sensitivity 4%, speed 4%, and time overhead 4%.

Figure 7 displays an Apache Storm that covers up to 17% of the area. The other tools include Apache Spark (11%), Python (11%), and C++-based frameworks (11%). The remaining tools include Microsoft Azure (6%), Occam (6%), Python (6%), RabitMQ (6%), RaspberryPi (6%), Spyder editor (6%), Apache Flink (5%), Docker container (5%), H2O (5%), and iFogsim (5%). For the measures listed in Figure 8 that are used for concept drift handling, accuracy is 24%. G-mean is 15%, recall is 13%, precision is 11%, f-measure is 11%, PMAUC is 4%, and each one from the remaining metrics

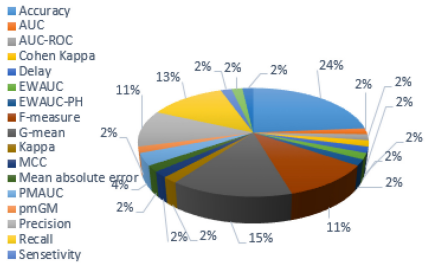

Fig. 9. Metrics used for concept drift handling.

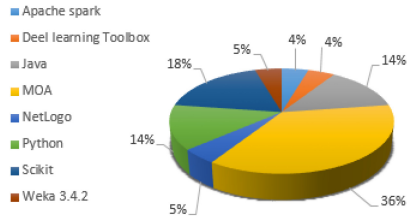

Fig. 10. Software tools used for concept drift handling.

occupies 2% area, according to the literature. Additionally, 36% of concept drift handling was done using the MOA simulator, 18% was done by using Scikit, 14% by using Java, 14% by using Python, 5% by using Weka 3.4.2, 5% by using Netlogo, 4% by using the Deep Learning Toolbox, and 4% was done by using Apache Storm, as shown in Figure 9.

6 Lessons Learned from Survey

Following are the lessons learned from the survey that is mentioned above: AMDO technique is the better solution for multi-class hybrid imbalanced datasets. The combined approach generates better results than the individual oversampling or undersampling techniques.

The imbalanced class problem is not the only aspect affecting the performance of the prediction model. Due to their high processing cost, high-dimensionality datasets have an impact on performance prediction. By removing the useless features, a few feature reduction-based classification models have been presented [51].

In concept drift issues, the dynamic integration has always been superior to the best base classifier and weighted voting, despite window shift or window size, and the learning algorithm.

Naive Bayes is mostly used as a prediction algorithm in retraining a model because of two reasons [60]:

— First, incremental learning is used, allowing the prediction model to be updated incrementally.
— Second, the computational complexity of Naive Bayes is rather low against the other methods of machine learning.

Online algorithms or incremental learning are the most appropriate and preferable methods for learning from massive amounts of data that are being processed in sequential steps [155].

Although incremental learning reduces complexity by simplifying the overall process through updating the model with new data without retraining from scratch, only a few machine learning methods (Naive Bayes, Neural Networks, and Hoeffding Trees) are capable of performing these incremental updates. Moreover, these incremental updates of models are unable to react to rapid changes that happen during the concept drifts.

The bulk of streaming data classification algorithms is either rules-based or tree-based to classify data. Ensemble, nearest neighbor, and statistical techniques are used in the development of very few algorithms. These findings show that there is still space for research in this field, as the performance of probability and machine learning–based categorization algorithms on streaming data remains an open research subject [156]. This survey discusses computational scalability in the context of Fog computing for handling imbalanced data streams. It explores the challenges related to computational scalability and emphasizes the need for lightweight techniques to address these issues effectively. Specifically, the survey mentions the importance of combining retraining learning

with incremental learning in Fog nodes using lightweight techniques to enhance computational scalability. Additionally, the survey highlights the constraints on memory, power, communication, and processing in Fog nodes, underscoring the significance of developing mechanisms that can handle imbalanced data streams efficiently while ensuring computational scalability.

7 Challenges and Future Directions

The major factors that are covered in this section are open questions, upcoming difficulties, and future research prospects for handling imbalanced data in Fog computing. The handling of multi-class imbalanced data streams on Fog nodes is a challenge due to constraints in memory, power, communication, and processing. A lot of time is required for the computations performed by resource-constrained devices at the edge. It is necessary to have a mechanism for handling imbalanced data streams that frequently update instances and forecasts unique as well as recurring classes. Investigating deep reinforcement learning and **generative adversarial networks (GANs)**, developing real-time lightweight and **Automated Machine Learning (AutoML)** systems for streaming data are required for handling concept drift and imbalanced data in Fog computing environments. Additionally, federated learning techniques, in which models are trained locally and then aggregated on a regular basis, may be used to create a robust global model that can adapt to new data and concept drift efficiently. Optimize the performance of fog computing systems by integrating edge AI and federated learning techniques to minimize data transfer and enhance local processing capabilities. Transfer learning and cross-domain adaptation need to be handled, ensuring that models trained in one domain can be used in another, especially when there are patterns or features shared by data streams from different domain. Explore hybrid approaches that combine data-level techniques with algorithm-level solutions to improve the robustness and accuracy of models in Fog computing. Implement hybrid Edge-Fog-Cloud architectures that leverage the strengths of each layer for optimized data processing and concept drift management.

To handle multi-class imbalanced data and concept drift in Fog nodes, develop algorithms that detect outliers and minimize their impact. Use privacy-preserving techniques to manage data without compromising confidentiality. Combine incremental learning, which updates models quickly, with retraining to adapt to sudden concept drift. This approach optimizes performance despite limited resources. Addressing these challenges collectively will improve data processing and decision-making in Fog environments.

8 Conclusion

A Fog computing and IoT network's performance can be considerably enhanced by effective batch and stream data processing approaches. No study has been identified in Fog computing on how to handle uneven data streams, but a significant amount of work has already been done with batch data. In this article, the researchers investigated the recent imbalanced data handling methods for processing batch and stream data in WSN, IoT networks, and Fog computing. Binary and multi-class imbalanced data are further subcategories of both types (batch and stream data) of data. Resampling, algorithmic, cost-sensitive, and ensemble are the four broad categories into which the various approaches are divided to treat imbalanced data. The present study has described the methodologies, their contributions, performance metrics, and tools of every approach. The analysis shows that, although ensemble learning is the preferred strategy, most researchers have used resampling strategies. The research results show that in 23% of the study, the MAUC was used as a popular metric for handling multi-class imbalanced data. In the case of IoT network, 22% of the studies used the accuracy metric for handling multi-class imbalanced data, and the research related to the assessment environment for concept drift reveals that accuracy was employed as

a common performance metric for concept drift handling in 24% of the studies, while 36% of the studies used the MOA as an optimization tool for concept drift handling. Moreover, the commonly used tool for stream processing in Fog was Apache Storm, which covered 17% of the area. These findings can be used for further research works.

References

[1] Shaik Masthan Babu, A. Jaya Lakshmi, and B. Thirumala Rao. 2015. A study on cloud based Internet of Things: CloudIoT. In *Global Conference on Communication Technologies (GCCT'15)*. IEEE, 60–65.

[2] Bushra Jamil, Humaira Ijaz, Mohammad Shojafar, Kashif Munir, and Rajkumar Buyya. 2022. Resource allocation and task scheduling in fog computing and internet of everything environments: A taxonomy, review, and future directions. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–38.

[3] Enkhtur Tsogbaatar, Monowar H. Bhuyan, Yuzo Taenaka, Doudou Fall, Khishigjargal Gonchigsumlaa, Erik Elmroth, and Youki Kadobayashi. 2021. DeL-IoT: A deep ensemble learning approach to uncover anomalies in IoT. *Internet Things* 14 (2021), 100391.

[4] A. Jaokar. 2016. *Data Science for Internet of Things (IoT): Ten Differences From Traditional Data Science*. KDnuggets.

[5] D. Friedman. 2015. *Data Science for Internet of Things (IoT): Ten Differences From Traditional Data Science*. ReadWrite. https://readwrite.com/five-types-data-internet-of-things/

[6] Alessio Botta, Walter De Donato, Valerio Persico, and Antonio Pescapé. 2016. Integration of cloud computing and Internet of Things: A survey. *Fut. Gen. Comput. Syst.* 56 (2016), 684–700.

[7] Flavio Bonomi, Rodolfo Milito, Jiang Zhu, and Sateesh Addepalli. 2012. Fog computing and its role in the Internet of Things. In *1st Edition of the MCC Workshop on Mobile Cloud Computing*. 13–16.

[8] P. PunithaIlayarani and M. Maria Dominic. 2019. Anatomization of fog computing and edge computing. In *IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT'19)*. IEEE, 1–6.

[9] Swati Malik and Kamali Gupta. 2019. Resource scheduling in fog: Taxonomy and related aspects. *J. Comput. Theoret. Nanosci.* 16, 10 (2019), 4313–4319.

[10] Aparna Kumari, Sudeep Tanwar, Sudhanshu Tyagi, Neeraj Kumar, Reza M. Parizi, and Kim-Kwang Raymond Choo. 2019. Fog data analytics: A taxonomy and process model. *J. Netw. Comput. Applic.* 128 (2019), 90–104.

[11] Ricardo Barandela, José Salvador Sánchez, Vicente Garcıa, and Edgar Rangel. 2003. Strategies for learning in class imbalance problems. *Pattern Recog.* 36, 3 (2003), 849–851.

[12] Pattaramon Vuttipittayamongkol and Eyad Elyan. 2020. Neighbourhood-based undersampling approach for handling imbalanced and overlapped data. *Inf. Sci.* 509 (2020), 47–70.

[13] Silvia Cateni, Valentina Colla, and Marco Vannucci. 2014. A method for resampling imbalanced datasets in binary classification tasks for real-world problems. *Neurocomputing* 135 (2014), 32–41.

[14] Wei-Chao Lin, Chih-Fong Tsai, Ya-Han Hu, and Jing-Shang Jhang. 2017. Clustering-based undersampling in class-imbalanced data. *Inf. Sci.* 409 (2017), 17–26.

[15] Debashree Devi, Suyel Namasudra, and Seifedine Kadry. 2020. A boosting-aided adaptive cluster-based undersampling approach for treatment of class imbalance problem. *Int. J. Data Warehous. Min.* 16, 3 (2020), 60–86.

[16] José Antonio Sanz, Dario Bernardo, Francisco Herrera, Humberto Bustince, and Hani Hagras. 2014. A compact evolutionary interval-valued fuzzy rule-based classification system for the modeling and prediction of real-world financial applications with imbalanced data. *IEEE Trans. Fuzzy Syst.* 23, 4 (2014), 973–990.

[17] Georgios Douzas, Fernando Bacao, and Felix Last. 2018. Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE. *Inf. Sci.* 465 (2018), 1–20.

[18] Lina Gong, Shujuan Jiang, and Li Jiang. 2019. Tackling class imbalance problem in software defect prediction through cluster-based over-sampling with filtering. *IEEE Access* 7 (2019), 145725–145737.

[19] György Kovács. 2019. An empirical comparison and evaluation of minority oversampling techniques on a large number of imbalanced datasets. *Appl. Soft Comput.* 83 (2019), 105662.

[20] Dech Thammasiri, Dursun Delen, Phayung Meesad, and Nihat Kasap. 2014. A critical assessment of imbalanced class distribution problem: The case of predicting freshmen student attrition. *Expert Syst. Applic.* 41, 2 (2014), 321–330.

[21] Pin Lim, Chi Keong Goh, and Kay Chen Tan. 2016. Evolutionary cluster-based synthetic oversampling ensemble (eco-ensemble) for imbalance learning. *IEEE Trans. Cybern.* 47, 9 (2016), 2850–2861.

[22] György Kovács. 2019. Smote-variants: A Python implementation of 85 minority oversampling techniques. *Neurocomputing* 366 (2019), 352–354.

[23] Dong-Sheng Cao, Qing-Song Xu, Yi-Zeng Liang, Liang-Xiao Zhang, and Hong-Dong Li. 2010. The boosting: A new idea of building models. *Chemomet. Intell. Lab. Syst.* 100, 1 (2010), 1–11.

[24] You-Shyang Chen. 2016. An empirical study of a hybrid imbalanced-class DT-RST classification procedure to elucidate therapeutic effects in uremia patients. *Med. Biol. Eng. Comput.* 54, 6 (2016), 983–1001.
[25] Sobhan Sarkar, Nikhil Khatedi, Anima Pramanik, and J. Maiti. 2020. An ensemble learning-based undersampling technique for handling class-imbalance problem. In *International Conference on Emerging Trends in Information Technology (ICETIT'19)*. Springer, 586–595.
[26] Guo Haixiang, Li Yijing, Jennifer Shang, Gu Mingyun, Huang Yuanyue, and Gong Bing. 2017. Learning from class-imbalanced data: Review of methods and applications. *Expert Syst. Applic.* 73 (2017), 220–239.
[27] Hengyi Wei, Baocheng Sun, and Mingming Jing. 2014. BalancedBoost: A hybrid approach for real-time network traffic classification. In *23rd International Conference on Computer Communication and Networks (ICCCN'14)*. IEEE, 1–6.
[28] José F. Díez-Pastor, Juan J. Rodríguez, Cesar Garcia-Osorio, and Ludmila I. Kuncheva. 2015. Random balance: Ensembles of variable priors classifiers for imbalanced data. *Knowl.-based Syst.* 85 (2015), 96–111.
[29] Lara Lusa and others. 2017. Gradient boosting for high-dimensional prediction of rare events. *Computational Statistics & Data Analysis* 113 (2017), 19–37.
[30] Sarah Vluymans, Isaac Triguero, Chris Cornelis, and Yvan Saeys. 2016. EPRENNID: An evolutionary prototype reduction based ensemble for nearest neighbor classification of imbalanced data. *Neurocomputing* 216 (2016), 596–610.
[31] Sergio Gónzalez, Salvador García, Marcelino Lázaro, Aníbal R. Figueiras-Vidal, and Francisco Herrera. 2017. Class switching according to nearest enemy distance for learning from highly imbalanced data-sets. *Pattern Recog.* 70 (2017), 12–24.
[32] Dan Gan, Jiang Shen, Bang An, Man Xu, and Na Liu. 2020. Integrating TANBN with cost sensitive classification algorithm for imbalanced data in medical diagnosis. *Comput. Industr. Eng.* 140 (2020), 106266.
[33] Salman H. Khan, Munawar Hayat, Mohammed Bennamoun, Ferdous A. Sohel, and Roberto Togneri. 2017. Cost-sensitive learning of deep feature representations from imbalanced data. *IEEE Trans. Neural Netw. Learn. Syst.* 29, 8 (2017), 3573–3587.
[34] Chong Zhang, Kay Chen Tan, and Ruoxu Ren. 2016. Training cost-sensitive deep belief networks on imbalance data problems. In *International Joint Conference on Neural Networks (IJCNN'16)*. IEEE, 4362–4367.
[35] Chong Zhang, Kay Chen Tan, Haizhou Li, and Geok Soon Hong. 2018. A cost-sensitive deep belief network for imbalanced classification. *IEEE Trans. Neural Netw. Learn. Syst.* 30, 1 (2018), 109–122.
[36] Weijie Zheng and Hong Zhao. 2020. Cost-sensitive hierarchical classification for imbalance classes. *Appl. Intell.* 50, 8 (2020), 2328–2338.
[37] Victoria López, Alberto Fernández, María José Del Jesus, and Francisco Herrera. 2013. A hierarchical genetic fuzzy system based on genetic programming for addressing classification with highly imbalanced and borderline data-sets. *Knowl.-based Syst.* 38 (2013), 85–104.
[38] Harshita Patel and Ghanshyam Singh Thakur. 2017. Classification of imbalanced data using a modified fuzzy-neighbor weighted approach. *Int. J. Intell. Eng. Syst.* 10, 1 (2017), 56–64.
[39] Harshita Patel and G. S. Thakur. 2019. An improved fuzzy k-nearest neighbor algorithm for imbalanced data using adaptive approach. *IETE J. Res.* 65, 6 (2019), 780–789.
[40] Maede Zolanvari, Marcio A. Teixeira, and Raj Jain. 2018. Effect of imbalanced datasets on security of industrial IoT using machine learning. In *IEEE International Conference on Intelligence and Security Informatics (ISI'18)*. IEEE, 112–117.
[41] Alberto Fernández, María José Del Jesus, and Francisco Herrera. 2010. Multi-class imbalanced data-sets with linguistic fuzzy rule based classification systems based on pairwise learning. In *International Conference on Information Processing and Management of Uncertainty in Knowledge-based Systems*. Springer, 89–98.
[42] Xuebing Yang, Qiuming Kuang, Wensheng Zhang, and Guoping Zhang. 2017. AMDO: An over-sampling technique for multi-class imbalanced problems. *IEEE Trans. Knowl. Data Eng.* 30, 9 (2017), 1672–1685.
[43] Zhongliang Zhang, Bartosz Krawczyk, Salvador Garcia, Alejandro Rosales-Pérez, and Francisco Herrera. 2016. Empowering one-vs-one decomposition with ensemble learning for multi-class imbalanced data. *Knowl.-based Syst.* 106 (2016), 251–263.
[44] Qianmu Li, Yanjun Song, Jing Zhang, and Victor S. Sheng. 2020. Multiclass imbalanced learning with one-versus-one decomposition and spectral clustering. *Expert Syst. Applic.* 147 (2020), 113152.
[45] Nutthaporn Junsomboon and Tanasanee Phienthrakul. 2017. Combining over-sampling and under-sampling techniques for imbalance dataset. In *9th International Conference on Machine Learning and Computing*. 243–247.
[46] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic minority over-sampling technique. *J. Arti. Intell. Res.* 16 (2002), 321–357.
[47] Barnan Das, Narayanan C. Krishnan, and Diane J. Cook. 2014. RACOG and wRACOG: Two probabilistic oversampling techniques. *IEEE Trans. Knowl. Data Eng.* 27, 1 (2014), 222–234.

[48] Lida Abdi and Sattar Hashemi. 2015. To combat multi-class imbalanced problems by means of over-sampling techniques. *IEEE Trans. Knowl. Data Eng.* 28, 1 (2015), 238–251.
[49] Show-Jane Yen and Yue-Shi Lee. 2009. Cluster-based under-sampling approaches for imbalanced data distributions. *Expert Syst. Applic.* 36, 3 (2009), 5718–5727.
[50] Hu Li, Peng Zou, Xiang Wang, and Rongze Xia. 2013. A new combination sampling method for imbalanced data. In *Chinese Intelligent Automation Conference*. Springer, 547–554.
[51] Ali Arshad, Saman Riaz, and Licheng Jiao. 2019. Semi-supervised deep fuzzy C-mean clustering for imbalanced multi-class classification. *IEEE Access* 7 (2019), 28100–28112.
[52] Tanapol Kosolwattana, Chenang Liu, Renjie Hu, Shizhong Han, Hua Chen, and Ying Lin. 2023. A self-inspected adaptive SMOTE algorithm (SASMOTE) for highly imbalanced data classification in healthcare. *BioData Min.* 16, 1 (2023), 15.
[53] Masoumeh Soleimani and Akram Sadat Mirshahzadeh. 2023. Multi-class classification of imbalanced intelligent data using deep neural network. *EAI Endors. Trans. AI Robot.* 2 (2023).
[54] Sukarna Barua, Md Monirul Islam, and Kazuyuki Murase. 2015. GOS-IL: A generalized over-sampling based online imbalanced learning framework. In *Neural Information Processing: 22nd International Conference, ICONIP 2015, Istanbul, Turkey, November 9-12, 2015, Proceedings, Part I 22*. Springer, 680–687.
[55] Shuo Wang, Leandro L. Minku, and Xin Yao. 2016. Dealing with multiple classes in online class imbalance learning. In *International Joint Conference on Artificial Intelligence (IJCAI'16)*. 2118–2124.
[56] Tahseen Al-Khateeb, Mohammad M. Masud, Khaled M. Al-Naami, Sadi Evren Seker, Ahmad M. Mustafa, Latifur Khan, Zouheir Trabelsi, Charu Aggarwal, and Jiawei Han. 2015. Recurring and novel class detection using class-based ensemble for evolving data stream. *IEEE Trans. Knowl. Data Eng.* 28, 10 (2015), 2752–2764.
[57] Zahraa S. Abdallah, Mohamed Medhat Gaber, Bala Srinivasan, and Shonali Krishnaswamy. 2016. AnyNovel: Detection of novel concepts in evolving data streams. *Evolv. Syst.* 7, 2 (2016), 73–93.
[58] Ahmad M. Mustafa, Gbadebo Ayoade, Khaled Al-Naami, Latifur Khan, Kevin W. Hamlen, Bhavani Thuraisingham, and Frederico Araujo. 2017. Unsupervised deep embedding for novel class detection over data stream. In *IEEE International Conference on Big Data (Big Data'17)*. IEEE, 1830–1839.
[59] Imen Khamassi, Moamar Sayed-Mouchaweh, Moez Hammami, and Khaled Ghédira. 2018. Discussion and review on evolving data streams and concept drift adapting. *Evolv. Syst.* 9, 1 (2018), 1–23.
[60] Lucas Baier, Josua Reimold, and Niklas Kühl. 2020. Handling concept drift for predictions in business process mining. In *IEEE 22nd Conference on Business Informatics (CBI'20)*. IEEE, 76–83.
[61] Manzoor Ahmed Hashmani, Syed Muslim Jameel, Mobashar Rehman, and Atsushi Inoue. 2020. Concept drift evolution in machine learning approaches: A systematic literature review. *Int. J. Smart Sens. Intell. Syst.* 13, 1 (2020), 1.
[62] Sheng Chen and Haibo He. 2009. SERA: Selectively recursive approach towards nonstationary imbalanced stream data mining. In *International Joint Conference on Neural Networks*. IEEE, 522–529.
[63] Farnaz Sadeghi and Herna L. Viktor. 2021. Online-MC-queue: Learning from imbalanced multi-class streams. In *3rd International Workshop on Learning with Imbalanced Domains: Theory and Applications*. PMLR, 21–34.
[64] Shuo Wang, Leandro L. Minku, and Xin Yao. 2018. A systematic study of online class imbalance learning with concept drift. *IEEE Trans. Neural Netw. Learn. Syst.* 29, 10 (2018), 4802–4821.
[65] S. Priya and R. Annie Uthra. 2021. RETRACTED ARTICLE: Comprehensive analysis for class imbalance data with concept drift using ensemble based classification. *J. Amb. Intell. Human. Comput.* 12, 5 (2021), 4943–4956.
[66] Roberto S. M. Barros, Danilo R. L. Cabral, Paulo M. Gonçalves Jr, and Silas G. T. C. Santos. 2017. RDDM: Reactive drift detection method. *Expert Syst. Applic.* 90 (2017), 344–355.
[67] Roberto Souto Maior Barros and Silas Garrido T. Carvalho Santos. 2018. A large-scale comparison of concept drift detectors. *Inf. Sci.* 451 (2018), 348–370.
[68] Tegjyot Singh Sethi and Mehmed Kantardzic. 2018. Handling adversarial concept drift in streaming data. *Expert Syst. Applic.* 97 (2018), 18–40.
[69] Moritz Heusinger, Christoph Raab, and Frank-Michael Schleif. 2022. Passive concept drift handling via variations of learning vector quantization. *Neural Computing and Applications* 34, 1 (2022), 89–100.
[70] Arif Budiman, Mohamad Ivan Fanany, and Chan Basaruddin. 2016. Adaptive convolutional ELM for concept drift handling in online stream data. *arXiv preprint arXiv:1610.02348* (2016).
[71] Tao Peng, Sana Sellami, and Omar Boucelma. 2019. IoT data imputation with incremental multiple linear regression. *Open J. Internet Things* 5, 1 (2019), 69–79.
[72] Shuo Wang and Leandro L. Minku. 2020. AUC estimation and concept drift detection for imbalanced data streams with multiple classes. In *International Joint Conference on Neural Networks (IJCNN'20)*. IEEE, 1–8.
[73] Łukasz Korycki and Bartosz Krawczyk. 2021. Concept drift detection from multi-class imbalanced data streams. In *IEEE 37th International Conference on Data Engineering (ICDE'21)*. IEEE, 1068–1079.

[74] S. Ancy and D. Paulraj. 2020. Handling imbalanced data with concept drift by applying dynamic sampling and ensemble classification model. *Computer Communications* 153 (2020), 553–560.
[75] Zeng Li, Wenchao Huang, Yan Xiong, Siqi Ren, and Tuanfei Zhu. 2020. Incremental learning imbalanced data streams with concept drift: The dynamic updated ensemble algorithm. *Knowl.-based Syst.* 195 (2020), 105694.
[76] Dariusz Brzezinski, Leandro L. Minku, Tomasz Pewinski, Jerzy Stefanowski, and Artur Szumaczuk. 2021. The impact of data difficulty factors on classification of imbalanced and concept drifting data streams. *Knowl. Inf. Syst.* 63, 6 (2021), 1429–1469.
[77] Gregory Ditzler and Robi Polikar. 2012. Incremental learning of concept drift from streaming imbalanced data. *IEEE Trans. Knowl. Data Eng.* 25, 10 (2012), 2283–2301.
[78] Jing Gao, Wei Fan, Jiawei Han, and Philip S. Yu. 2007. A general framework for mining concept-drifting data streams with skewed distributions. In *Siam International Conference on Data Mining*. SIAM, 3–14.
[79] Josephine Akosa. 2017. Predictive accuracy: A misleading performance measure for highly imbalanced data. In *SAS Global Forum*.
[80] Hung Cao and Monica Wachowicz. 2019. An edge-fog-cloud architecture of streaming analytics for Internet of Things applications. *Sensors* 19, 16 (2019), 3594.
[81] Mercedes E. Paoletti, Oscar Mogollon-Gutierrez, Sergio Moreno-Álvarez, Jose Carlos Sancho, and Juan M. Haut. 2023. A comprehensive survey of imbalance correction techniques for hyperspectral data classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 16 (2023), 5297–5314.
[82] Debashree Devi, Saroj K. Biswas, and Biswajit Purkayastha. 2020. A review on solution to class imbalance problem: Undersampling approaches. In *International Conference on Computational Performance Evaluation (ComPE'20)*. IEEE, 626–631.
[83] Sanchita Pandey and Kuldeep Kumar. 2023. Software fault prediction for imbalanced data: A survey on recent developments. *Proced. Comput. Sci.* 218 (2023), 1815–1824.
[84] Abhisar Sharma, Anuradha Purohit, and Himani Mishra. 2021. A survey on imbalanced data handling techniques for classification. *Int. J. Emerg. Trends Eng. Res.* 9, 10 (2021).
[85] Shaik Johny Basha, Srinivasa Rao Madala, Kolla Vivek, Eedupalli Sai Kumar, and Tamminina Ammannamma. 2022. A review on imbalanced data classification techniques. In *International Conference on Advanced Computing Technologies and Applications (ICACTA'22)*. IEEE, 1–6.
[86] Megha Ashok Patil, Sunil Kumar, Sandeep Kumar, and Muskan Garg. 2021. Concept drift detection for social media: A survey. In *3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N'21)*. IEEE, 12–16.
[87] Meng Han, Zhiqiang Chen, Muhang Li, Hongxin Wu, and Xilong Zhang. 2022. A survey of active and passive concept drift handling methods. *Comput. Intell.* 38, 4 (2022), 1492–1535.
[88] Jiguo Yu, Yingying Qi, Guanghui Wang, and Xin Gu. 2012. A cluster-based routing protocol for wireless sensor networks with nonuniform node distribution. *AEU-Int. J. Electron. Commun.* 66, 1 (2012), 54–61.
[89] Nawel Yala, Belkacem Fergani, Laurent Clavier, and others. 2014. Soft margin SVM modeling for handling imbalanced human activity datasets in multiple homes. In *2014 International Conference on Multimedia Computing and Systems (ICMCS)*. IEEE, 421–426.
[90] Hong Zhou and Kun-Ming Yu. 2017. Imbalanced data classification for defective product prediction based on industrial wireless sensor network. In *6th International Conference on Future Generation Communication Technologies (FGCT'17)*. IEEE, 1–6.
[91] Hang Yang, Simon Fong, Raymond Wong, and Guangmin Sun. 2013. Optimizing classification decision trees by using weighted naïve Bayes predictors to reduce the imbalanced class problem in wireless sensor network. *Int. J. Distrib. Sensor Netw.* 9, 1 (2013), 460641.
[92] Harshita Patel, Dharmendra Singh Rajput, G. Thippa Reddy, Celestine Iwendi, Ali Kashif Bashir, and Ohyun Jo. 2020. A review on classification of imbalanced data for wireless sensor networks. *Int. J. Distrib. Sensor Netw.* 16, 4 (2020), 1550147720916404.
[93] Sireesha Rodda and Uma Shankar Rao Erothi. 2016. Class imbalance problem in the network intrusion detection systems. In *International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT'16)*. IEEE, 2685–2688.
[94] Sitaram Asur and Srinivasan Parthasarathy. 2007. Correlation-based feature partitioning for rare event detection in wireless sensor networks. In *1st International Workshop on Knowledge Discovery from Sensor Data (Sensor-KDD'07)*.
[95] B. Abidine M'hamed and Belkacem Fergani. 2014. A new multi-class WSVM classification to imbalanced human activity dataset. *J. Comput.* 9, 7 (2014), 1560–1565.
[96] Yujing Chen, Zheng Chai, Yue Cheng, and Huzefa Rangwala. 2021. Asynchronous federated learning for sensor data with concept drift. In *IEEE International Conference on Big Data (Big Data'21)*. IEEE, 4822–4831.

[97] Shenglan Liu, Lin Feng, Jun Wu, Gang Hou, and Guangjie Han. 2017. Concept drift detection for data stream learning based on angle optimized global embedding and principal component analysis in sensor networks. *Comput. Electric. Eng.* 58 (2017), 327–336.

[98] Evangelos Bagkis, Theodosios Kassandros, and Kostas Karatzas. 2022. Learning calibration functions on the fly: Hybrid batch online stacking ensembles for the calibration of low-cost air quality sensor networks in the presence of concept drift. *Atmosphere* 13, 3 (2022), 416.

[99] Swaraj Dube, Wong Yee Wan, and Hermawan Nugroho. 2021. A novel approach of IoT stream sampling and model update on the IoT edge device for class incremental learning in an edge-cloud system. *IEEE Access* 9 (2021), 29180–29199.

[100] Akbar Telikani and Amir H. Gandomi. 2021. Cost-sensitive stacked auto-encoders for intrusion detection in the Internet of Things. *Internet Things* 14 (2021), 100122.

[101] Bharath Sudharsan, John G. Breslin, and Muhammad Intizar Ali. 2021. Imbal-OL: Online machine learning from imbalanced data streams in real-world IoT. In *IEEE International Conference on Big Data (Big Data'21)*. IEEE, 4974–4978.

[102] Gaurav Mohindru, Koushik Mondal, and Haider Banka. 2021. Different hybrid machine intelligence techniques for handling IoT-based imbalanced data. *CAAI Trans. Intell. Technol.* 6, 4 (2021), 405–416.

[103] Razan Abdulhammed, Miad Faezipour, Abdelshakour Abuzneid, and Arafat AbuMallouh. 2018. Deep and machine learning approaches for anomaly-based intrusion detection of imbalanced network traffic. *IEEE Sensors Lett.* 3, 1 (2018), 1–4.

[104] Mohamed Amine Ferrag, Leandros Maglaras, Ahmed Ahmim, Makhlouf Derdour, and Helge Janicke. 2020. RDTIDS: Rules and decision tree-based intrusion detection system for internet-of-things networks. *Fut. Internet* 12, 3 (2020), 44.

[105] JooHwa Lee and KeeHyun Park. 2021. GAN-based imbalanced data intrusion detection system. *Person. Ubiq. Comput.* 25, 1 (2021), 121–128.

[106] Chun-Cheng Lin, Der-Jiunn Deng, Chin-Hung Kuo, and Linnan Chen. 2019. Concept drift detection and adaption in big imbalance industrial IoT data using an ensemble learning method of offline classifiers. *IEEE Access* 7 (2019), 56198–56207.

[107] Sikha Bagui and Kunqi Li. 2021. Resampling imbalanced data for network intrusion detection datasets. *J. Big Data* 8, 1 (2021), 1–41.

[108] Pandit Byomakesha Dash, Janmenjoy Nayak, Bighnaraj Naik, Etuari Oram, and S. K. Hafizul Islam. 2020. Model based IoT security framework using multiclass adaptive boosting with SMOTE. *Secur. Privac.* 3, 5 (2020), e112.

[109] Pankaj Kumar Keserwani, Mahesh Chandra Govil, Emmanuel S. Pilli, and Prajjval Govil. 2021. A smart anomaly-based intrusion detection system for the Internet of Things (IoT) network using GWO–PSO–RF model. *J. Reliab. Intell. Environ.* 7, 1 (2021), 3–21.

[110] Mashaal A. Alfhaid and Manal A. Abdullah. 2022. ICSM: Imbalanced chunk-based stream model. *Int. J. Innov., Creativ. Change* 16 (2022).

[111] Vinay Arora, Rohan Singh Leekha, Kyungroul Lee, and Aman Kataria. 2020. Facilitating user authorization from imbalanced data logs of credit cards using artificial intelligence. *Mobile Information Systems* 2020, 1 (2020), 8885269.

[112] Siqi Ren, Bo Liao, Wen Zhu, Zeng Li, Wei Liu, and Keqin Li. 2018. The gradual resampling ensemble for mining imbalanced data streams with concept drift. *Neurocomputing* 286 (2018), 150–166.

[113] Rafiq Ahmed Mohammed, Kok-Wai Wong, Mohd Fairuz Shiratuddin, and Xuequn Wang. 2020. PWIDB: A framework for learning to classify imbalanced data streams with incremental data re-balancing technique. *Proced. Comput. Sci.* 176 (2020), 818–827.

[114] Li Yang, Dimitrios Michael Manias, and Abdallah Shami. 2021. PWPAE: An ensemble framework for concept drift adaptation in IoT data streams. In *IEEE Global Communications Conference (GLOBECOM'21)*. IEEE, 01–06.

[115] Asim Roy. 2016. Two-layered ensemble Kohonen nets for imbalanced streaming data. In *IEEE Congress on Evolutionary Computation (CEC'16)*. IEEE, 5215–5221.

[116] Seba Susan and Amitesh Kumar. 2021. The balancing trick: Optimized sampling of imbalanced datasets–A brief survey of the recent state of the art. *Eng. Rep.* 3, 4 (2021), e12298.

[117] Amin Shahraki, Mahmoud Abbasi, Amir Taherkordi, and Anca Delia Jurcut. 2022. A comparative study on online machine learning techniques for network traffic streams analysis. *Comput. Netw.* 207 (2022), 108836.

[118] Hongle Du, Yan Zhang, Ke Gang, Lin Zhang, and Yeh-Cheng Chen. 2021. Online ensemble learning algorithm for imbalanced data stream. *Applied Soft Computing* 107 (2021), 107378. DOI: https://doi.org/10.1016/j.asoc.2021.107378

[119] Manal Almuammar and Maria Fasli. 2018. Learning patterns from imbalanced evolving data streams. In *IEEE International Conference on Big Data (Big Data'18)*. IEEE, 2048–2057.

[120] Yange Sun, Yi Sun, and Honghua Dai. 2020. Two-stage cost-sensitive learning for data streams with concept drift and class imbalance. *IEEE Access* 8 (2020), 191942–191955.

[121] Debanjan Borthakur, Harishchandra Dubey, Nicholas Constant, Leslie Mahler, and Kunal Mankodiya. 2017. Smart fog: Fog computing framework for unsupervised clustering analytics in wearable Internet of Things. In *IEEE Global Conference on Signal and Information Processing (GlobalSIP'17)*. IEEE, 472–476.
[122] Branka Mikavica and Aleksandra Kostić-Ljubisavljević. 2019. *Fog Computing in Logistics Systems*. Logic.
[123] Yohan Joo, Jaehyeong Lee, and Jongpil Jeong. 2020. Ensemble fog computing architecture for unstable state detection of hydraulic system. *Proced. Comput. Sci.* 175 (2020), 230–236.
[124] Mohammed Anis Benblidia, Bouziane Brik, Leila Merghem-Boulahia, and Moez Esseghir. 2019. Ranking fog nodes for tasks scheduling in fog-cloud environments: A fuzzy logic approach. In *15th International Wireless Communications & Mobile Computing Conference (IWCMC'19)*. IEEE, 1451–1457.
[125] Arman Anzanpour, Humayun Rashid, Amir M. Rahmani, Axel Jantsch, Nikil Dutt, and Pasi Liljeberg. 2019. Energy-efficient and reliable wearable Internet-of-Things through fog-assisted dynamic goal management. *Proced. Comput. Sci.* 151 (2019), 493–500.
[126] Shengjie Xu, Yi Qian, and Rose Qingyang Hu. 2019. A semi-supervised learning approach for network anomaly detection in fog computing. In *IEEE International Conference on Communications (ICC'19)*. IEEE, 1–6.
[127] Saurabh Shukla, Mohd Fadzil Hassan, Muhammad Khalid Khan, Low Tang Jung, and Azlan Awang. 2019. An analytical model to minimize the latency in healthcare internet-of-things in fog computing environment. *PLoS One* 14, 11 (2019), e0224934.
[128] Saurabh Shukla, Mohd Fadzil Hassan, Low Tang Jung, Azlan Awang, and Muhammad Khalid Khan. 2019. A 3-tier architecture for network latency reduction in healthcare internet-of-things using fog computing and machine learning. In *8th International Conference on Software and Computer Applications*. 522–528.
[129] Jakub Jalowiczor, Jan Rozhon, and Miroslav Voznak. 2021. Study of the efficiency of fog computing in an optimized LoRaWAN cloud architecture. *Sensors* 21, 9 (2021), 3159.
[130] Nader Mohamed, Jameela Al-Jaroodi, Sanja Lazarova-Molnar, and Imad Jawhar. 2021. Applications of integrated IoT-fog-cloud systems to smart cities: A survey. *Electronics* 10, 23 (2021), 2918.
[131] AIA Alzahrani, A. Al-Rasheed, A. Ksibi, M. Ayadi, M. M. Asiri, and M. Zakariah. 2022. Anomaly detection in fog computing architectures using custom tab transformer for internet of things. *Electronics* 11, 23 (2022), 4017.
[132] Shusen Yang. 2017. IoT stream processing and analytics in the fog. *IEEE Commun. Mag.* 55, 8 (2017), 21–27.
[133] Shaimaa A. Hussein, Ahmed I. Saleh, Hossam El-Din Mostafa, and Marwa I. Obay. 2021. *A Hybrid Security Strategy (HS2) for Reliable Video Streaming in Fog Computing (Retraction of Vol 51, art no 102412, 2020)*. Elsevier Radarweg 29, 1043 NX Amsterdam, Netherlands.
[134] Jyun-Yao Huang, Wei-Chih Hong, Po-Shin Tsai, and I-En Liao. 2017. A model for aggregation and filtering on encrypted XML streams in fog computing. *Int. J. Distrib. Sensor Netw.* 13, 5 (2017), 1550147717704158.
[135] Brenno M. Alencar, Ricardo A. Rios, Cleber Santana, and Cássio Prazeres. 2020. FoT-Stream: A fog platform for data stream analytics in IoT. *Comput. Commun.* 164 (2020), 77–87.
[136] Elarbi Badidi and Karima Moumane. 2019. Enhancing the processing of healthcare data streams using fog computing. In *IEEE Symposium on Computers and Communications (ISCC'19)*. IEEE, 1113–1118.
[137] Paulo Bezerra, Adalberto Melo, Allan Douglas, Hugo Santos, Denis Rosário, and Eduardo Cerqueira. 2019. A collaborative routing protocol for video streaming with fog computing in vehicular ad hoc networks. *Int. J. Distrib. Sensor Netw.* 15, 3 (2019), 1550147719832839.
[138] Ivan Merelli, Lucia Morganti, Elena Corni, Carmelo Pellegrino, Daniele Cesini, Luca Roverelli, Gabriele Zereik, and Daniele D'Agostino. 2018. Low-power portable devices for metagenomics analysis: Fog computing makes bioinformatics ready for the Internet of Things. *Fut. Gen. Comput. Syst.* 88 (2018), 467–478.
[139] Juan Luis Pérez, Alberto Gutierrez-Torre, Josep Lluís Berral, and David Carrera. 2018. A resilient and distributed near real-time traffic forecasting application for Fog computing environments. *Fut. Gen. Comput. Syst.* 87 (2018), 198–212.
[140] Maria A. Rodriguez, Ramamohanarao Kotagiri, and Rajkumar Buyya. 2018. Detecting performance anomalies in scientific workflows using hierarchical temporal memory. *Fut. Gen. Comput. Syst.* 88 (2018), 624–635.
[141] Asif Muhammad and Muhammad Aleem. 2021. A3-Storm: Topology-, traffic-, and resource-aware storm scheduler for heterogeneous clusters. *J. Supercomput.* 77, 2 (2021), 1059–1093.
[142] Daniele De Sensi, Tiziano De Matteis, and Marco Danelutto. 2018. Simplifying self-adaptive and power-aware computing with Nornir. *Fut. Gen. Comput. Syst.* 87 (2018), 136–151.
[143] Ivan Walulya, Dimitris Palyvos-Giannas, Yiannis Nikolakopoulos, Vincenzo Gulisano, Marina Papatriantafilou, and Philippas Tsigas. 2018. Viper: A module for communication-layer determinism and scaling in low-latency stream processing. *Fut. Gen. Comput. Syst.* 88 (2018), 297–308.
[144] Valeria Cardellini, Francesco Lo Presti, Matteo Nardelli, and Gabriele Russo Russo. 2018. Decentralized self-adaptation for elastic data stream processing. *Fut. Gen. Comput. Syst.* 87 (2018), 171–185.
[145] Claudia Misale, Maurizio Drocco, Guy Tremblay, Alberto R. Martinelli, and Marco Aldinucci. 2018. PiCo: High-performance data analytics pipelines in modern C++. *Fut. Gen. Comput. Syst.* 87 (2018), 392–403.

[146] Zoltán Zvara, Péter G. N. Szabó, Barnabás Balázs, and András Benczúr. 2019. Optimizing distributed data stream processing by tracing. *Fut. Gen. Comput. Syst.* 90 (2019), 578–591.
[147] Arvind Kumar Gangwar and Sandeep Kumar. 2023. Concept drift in software defect prediction: A method for detecting and handling the drift. *ACM Trans. Internet Technol.* 23, 2 (2023), 1–28.
[148] Ature Angbera and Huah Yong Chan. 2024. An adaptive XGBoost-based optimized sliding window for concept drift handling in non-stationary spatiotemporal data streams classifications. *J. Supercomput.* 80, 6 (2024), 7781–7811.
[149] Ketan Sanjay Desale and Swati V. Shinde. 2023. Concept drift detection and adaption framework using optimized deep learning and adaptive sliding window approach. *Expert Syst.* 40, 9 (2023), e13394.
[150] Tao Huang, Boyi Xu, Hongming Cai, Jiawei Du, Kuo-Ming Chao, and Chengxi Huang. 2018. A fog computing based concept drift adaptive process mining framework for mobile APPs. *Fut. Gen. Comput. Syst.* 89 (2018), 670–684.
[151] Brenno M. Alencar, João Paulo Canário, Ruivaldo Lobão Neto, Cássio Prazeres, Abert Bifet, and Ricardo A. Rios. 2023. Fog-DeepStream: A new approach combining LSTM and concept drift for data stream analytics on Fog computing. *Internet Things* 22 (2023), 100731.
[152] Amir Hossein Estiri and Muthucumaru Maheswaran. 2021. Attentive federated learning for concept drift in distributed 5G edge networks. *arXiv preprint arXiv:2111.07457* (2021).
[153] Hassan Mehmood, Panos Kostakos, Marta Cortes, Theodoros Anagnostopoulos, Susanna Pirttikangas, and Ekaterina Gilman. 2021. Concept drift adaptation techniques in distributed environment for real-world data streams. *Smart Cities* 4, 1 (2021), 349–371.
[154] I. Made Murwantara and Pujianto Yugopuspito. 2021. An adaptive IoT architecture using combination of concept-drift and dynamic software product line engineering. *TELKOMNIKA (Telecommun. Comput. Electron. Contr.)* 19, 4 (2021), 1226–1233.
[155] Pallavi Kulkarni and Roshani Ade. 2014. Incremental learning from unbalanced data with concept class, concept drift and missing features: A review. *Int. J. Data Min. Knowl. Manag. Process* 4, 6 (2014), 15.
[156] Shikha Mehta and others. 2017. Concept drift in streaming data classification: algorithms, platforms and issues. *Procedia Computer Science* 122 (2017), 804–811.

Received 28 January 2023; revised 13 July 2024; accepted 6 August 2024